Learning Complex Physical Systems

Francesco Cenni Francesco Giordani Giulio Mazzaroppi Giacomo Pezzolati

University of Bologna

AI in Industry

francesco.cenni3@studio.unibo.it
francesco.giordani5@studio.unibo.it
giulio.mazzaroppi@studio.unibo.it
giacomo.pezzolati@studio.unibo.it

December 17, 2025

Overview

1. Introduction & Motivation
2. Modeling and data generation
3. Data Inspection
4. Black Box Approaches
5. Parameter Identification
6. Naive Approaches
7. Neural Networks with Physical Structure
8. Neuro Probabilistic Model
9. Conclusions and Future Works
10. Appendix

Idea of the project

- **Problem context**: modeling non-linear dynamical systems
- **Why a double pendulum**: well-known, relatively simple dynamics, chaotic behavior
- **Industrial relevance**:
  - Benchmark for ML & DL for physical systems
  - Applicable to more complex fields (e.g. **robotics**, **aerospace**)
- **Applications**: control of multi-articulated systems, energy optimization, short-term prediction

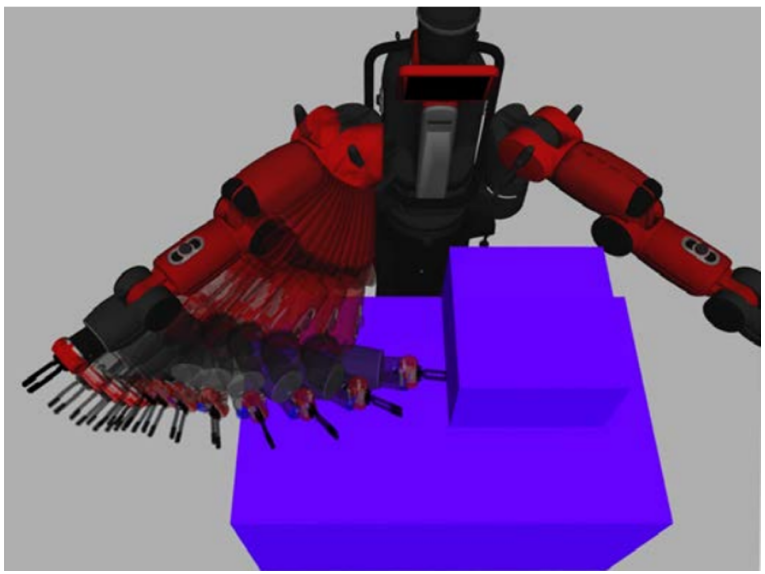


Figure: Robotic manipulator motion control

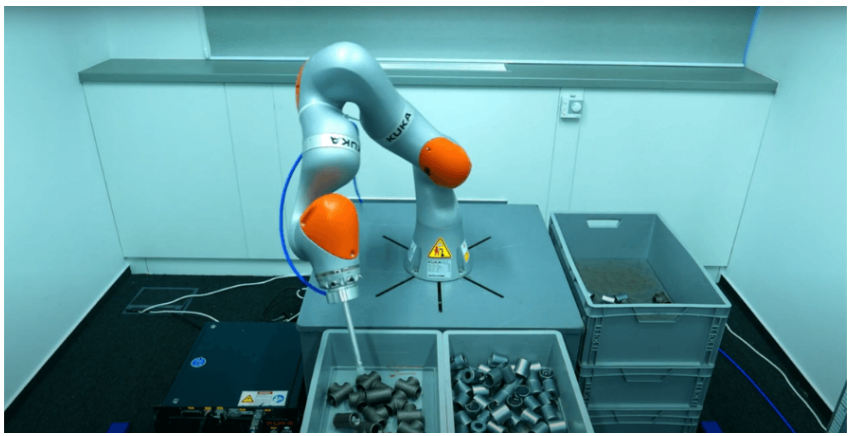


Figure: Impedance control

Double Pendulum Setup

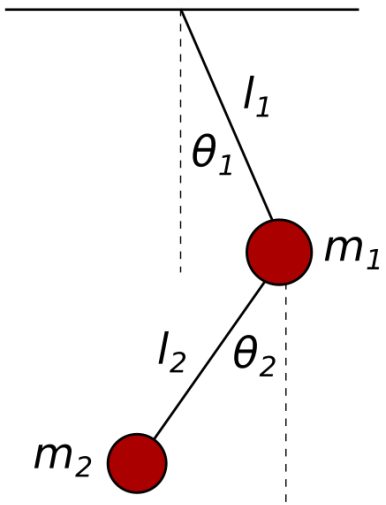


Figure: Double Pendulum Model

State Variables

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} \in \mathbb{R}^4$$

Where:

- $\begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \in \mathbb{R}^2$ are the angular positions
- $\begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \in \mathbb{R}^2$ are the angular velocities

Double Pendulum Dynamics

From the Euler-Lagrange equations:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} + \frac{\partial \mathcal{R}}{\partial \dot{q}_i} = \tau_i, \quad i = 1, 2 \tag{1}$$

where $\mathcal{L} = T - V$ is the Lagrangian

- T is the kinetic energy
- V is the potential energy
- R is the Rayleigh dissipation function
- τ_i are the generalized forces (torques) applied at the joints

The dynamic model of the double pendulum can be expressed in the following matrix form[1]

Dynamics in Matrix Form

$$M(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + D\dot{\mathbf{q}} + G(\mathbf{q}) = \tau \tag{2}$$

[1] Explicit expression reported in the appendix A

Data Generation

Integration:

- **Forward Euler:**
  - ✓ simple
  - × less accurate
- **Runge-Kutta 4th Order (RK4):**
  - ✓ more accurate
  - × computationally expensive

Generation

Setting:

- Duration $T = 10$ s with step size $\Delta t = 0.01$ s
- Initial state $q_0 \sim \mathcal{U}[-\pi, \pi], \dot{q}_0 \sim \mathcal{U}[-2, 2]$
- Physical parameters m_1, m_2, l_1, l_2, g(gravity), b_1, b_2(friction)

Datsets:

- **No friction** $b_1 = b_2 = 0$
- **Low friction** $b_1 = b_2 = 0.3$
- **High friction** $b_1 = b_2 = 3.0$ *Excluded for rapid energy dissipation*

Motivation for Learning Dynamics

Why do we need data-driven approaches?

- **Complexity**: real physical systems are chaotic and highly non-linear
- **Cost**: accurate parameter estimation via sensors is expensive and prone to noise
- **Adaptability**: analytical models fail when system properties change over time (e.g., wear and tear)

Data Inspection

Why is Data Inspection crucial?
Before training any model, we must validate the physical consistency and statistical quality of our simulation data

Our Inspection Pipeline:

1. **Physics Validation:** ensuring energy conservation laws hold (for frictionless cases)
2. **Distribution Analysis:** checking for class imbalance or skewness in state variables
3. **Stationarity Check:** verifying that statistical properties (mean, variance) remain stable over time
4. **Correlation Study:** identifying redundant features to simplify the learning task

Pre-processing

Based on the data inspection results, a preprocessing pipeline was designed to enhance data quality and model training effectiveness

- **Filtering out evolution with increasing energy**: removing unstable or numerically faulty simulations
- **White Noise**: added noise to simulate reality and add robustness to the system
- **KL Divergence + Cut**:
  - Calculation of the **Kullback-Leibler Divergence** between actual distribution and uniform distribution. Cut of the dataset at the optimum threshold K^* that minimizes the divergence
  - Identification and removal of the part of the dataset in which the dynamics is less informative

Data Preprocessing: Dynamic Time Cut Inspection

Following the KL Divergence analysis, the dataset was trimmed to exclude less informative dynamics, enhancing model training quality

Kullback-Leibler Divergence Definition

The Kullback-Leibler Divergence between two probability distributions P and Q is defined as:

$$D_{KL}(P||Q) = \sum_i P(i) \log\left(\frac{P(i)}{Q(i)}\right) \tag{3}$$

Where:

- Q observed distribution
- $P \sim \mathcal{U}$ uniform distribution

Adaptive Dataset Truncation (1)

Problem: In high-friction scenarios, the system eventually settles into a static equilibrium. Including these static frames would bias the model towards zero-velocity predictions

Solution: We treat truncation as an optimization problem to find the optimal cutoff time T^*

The Objective Function

We minimize a custom loss function combining **distribution quality** and **data retention**:

$$\mathcal{L}(T) = \underbrace{D_{KL}(P||Q_T)}_{\text{Chaos Metric}} + \lambda \cdot \underbrace{\left(\frac{T_{max} - T}{T_{max}}\right)^2}_{\text{Data Loss Penalty}} \tag{4}$$

$$T^* = \arg\min_T \mathcal{L}(T) \tag{5}$$

Adaptive Dataset Truncation (2)

Where:

- Q_T: observed distribution up to time T
- T_{max}: total simulation time
- λ: regularization parameter balancing the two terms
- T: candidate truncation time

Result: The algorithm automatically identifies the exact moment the system loses its ergodic properties

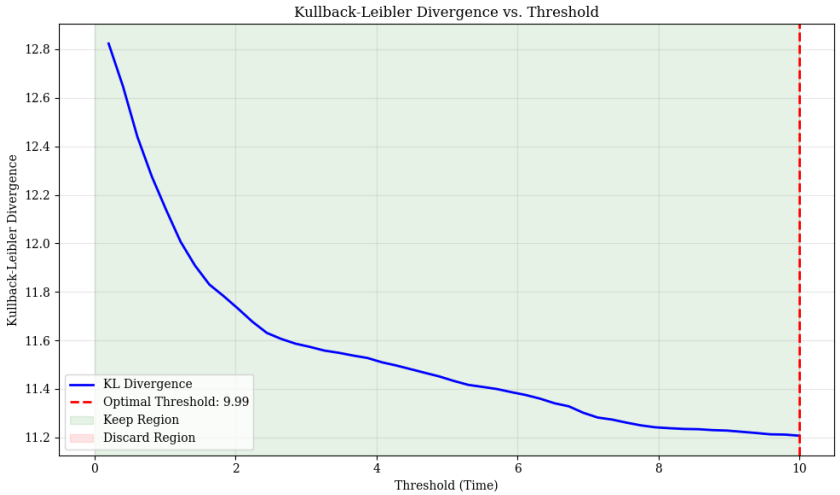
Recall: Ergodicity Hypothesis

A dynamical system is **Ergodic** if it visits every point in the phase space with equal probability over a long period

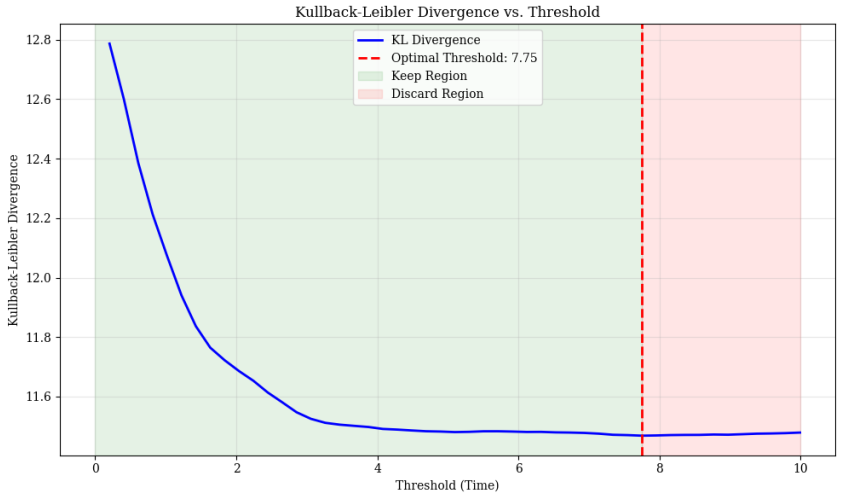
$$\text{Time Average} \equiv \text{Space Average}$$

In our context: Ideally, a frictionless pendulum explores all possible angles and velocities allowed by its energy, resulting in a **Uniform Distribution** (P)

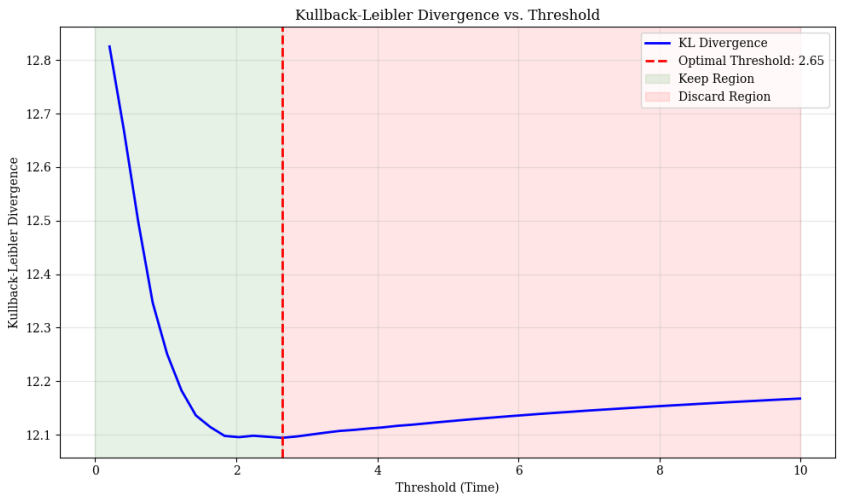
Data Truncation Results



No Friction

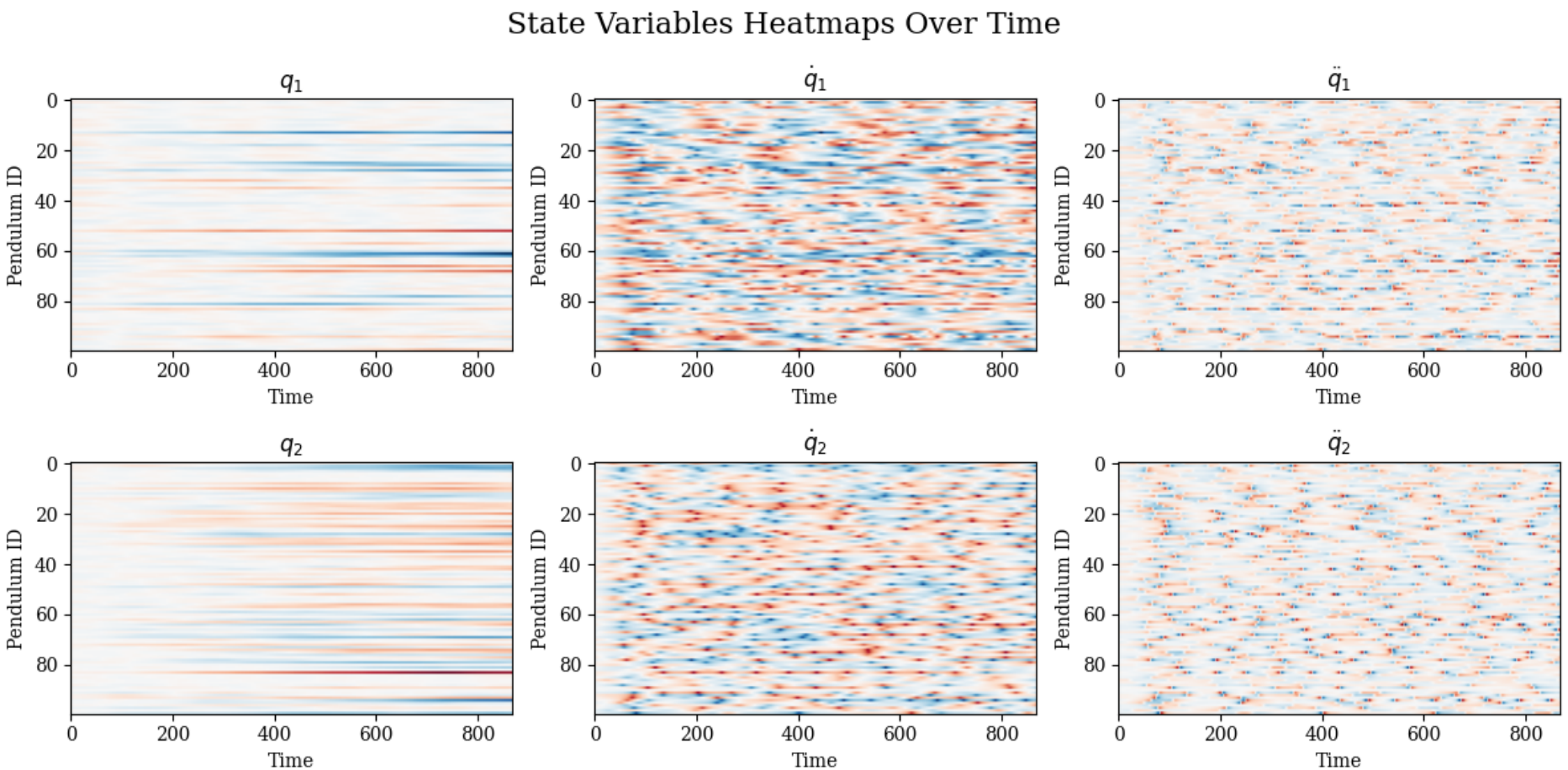


Friction



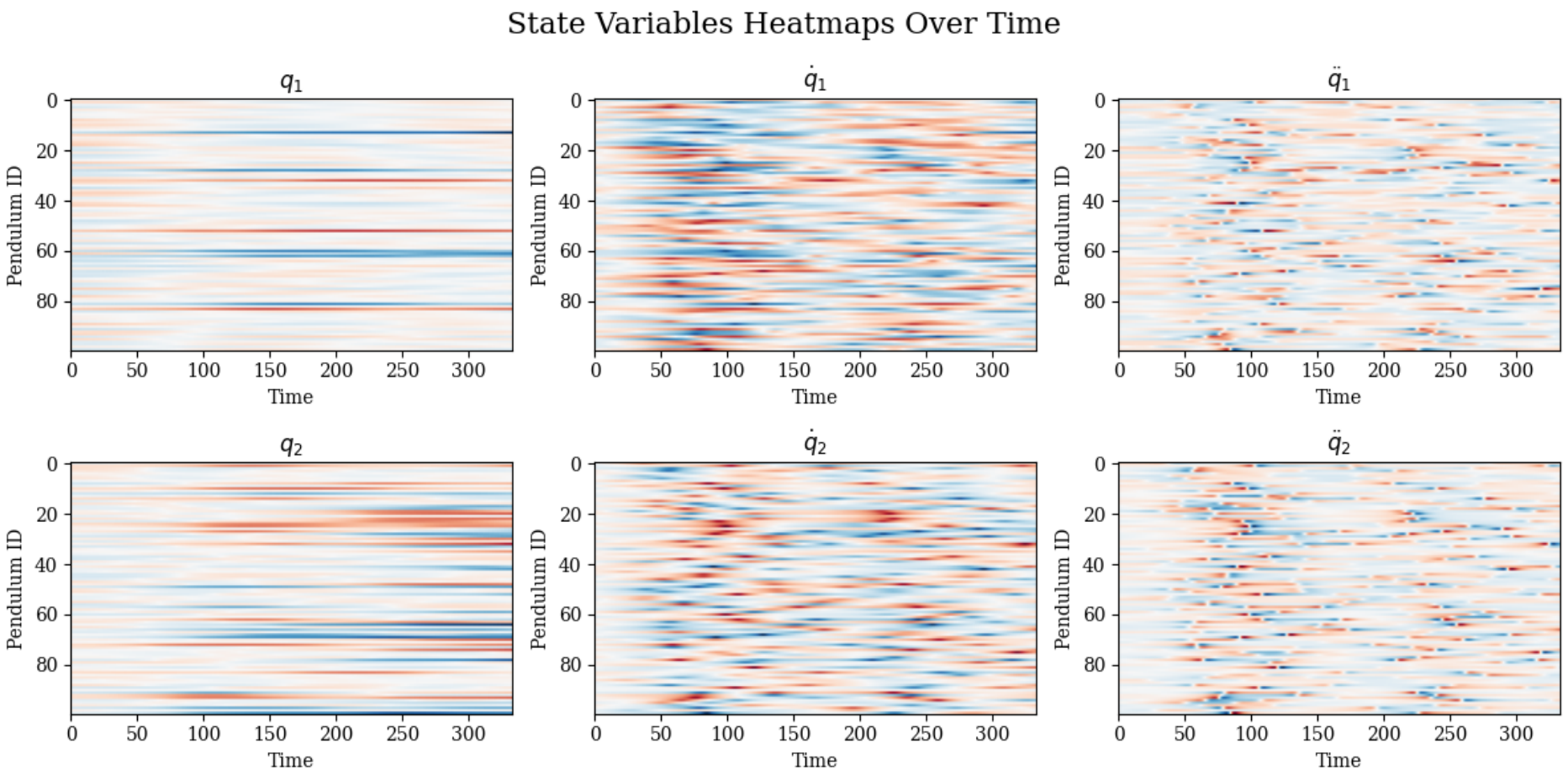
High Friction

Heatmap (1)



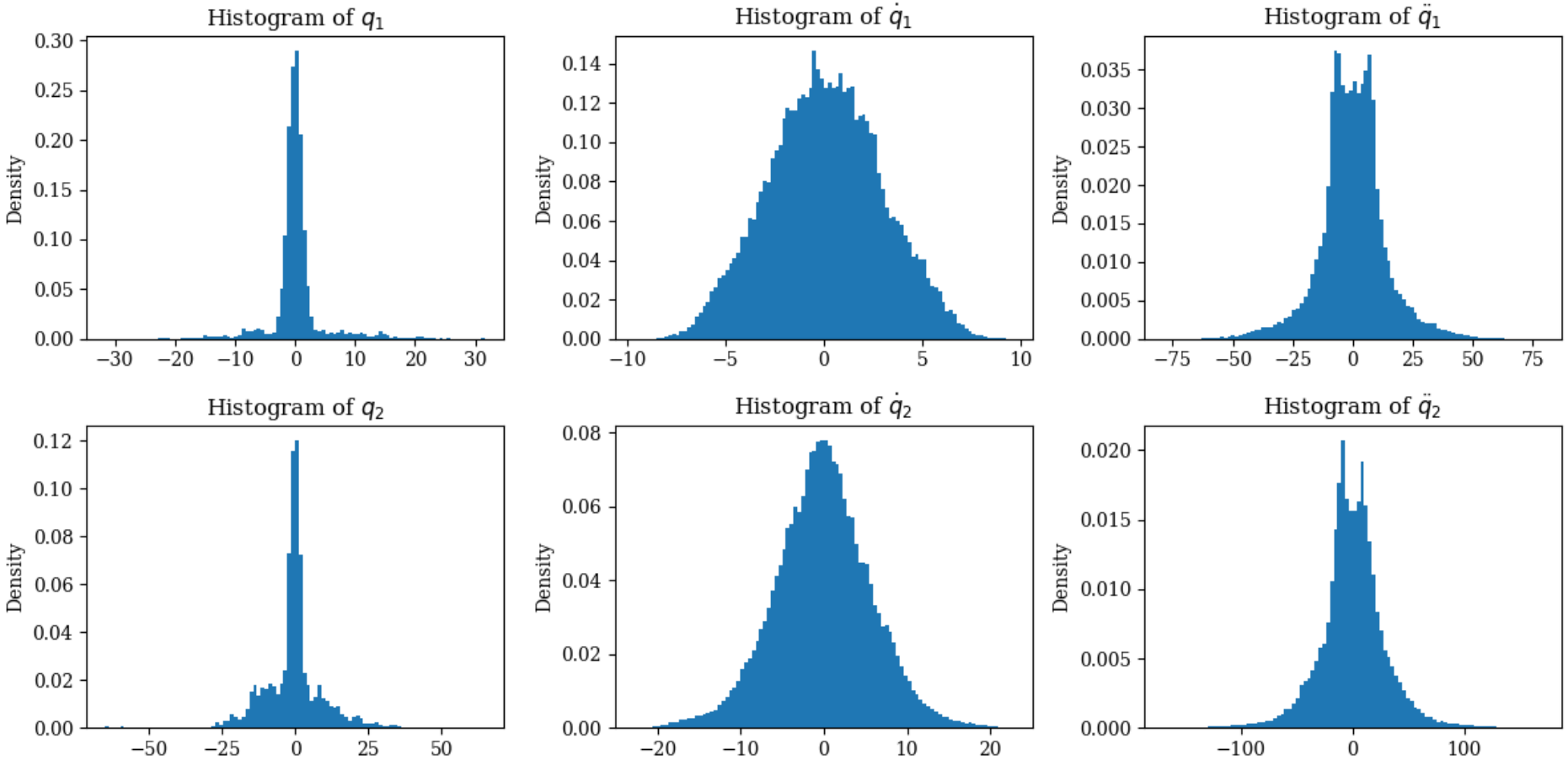
No Friction

Heatmap (2)



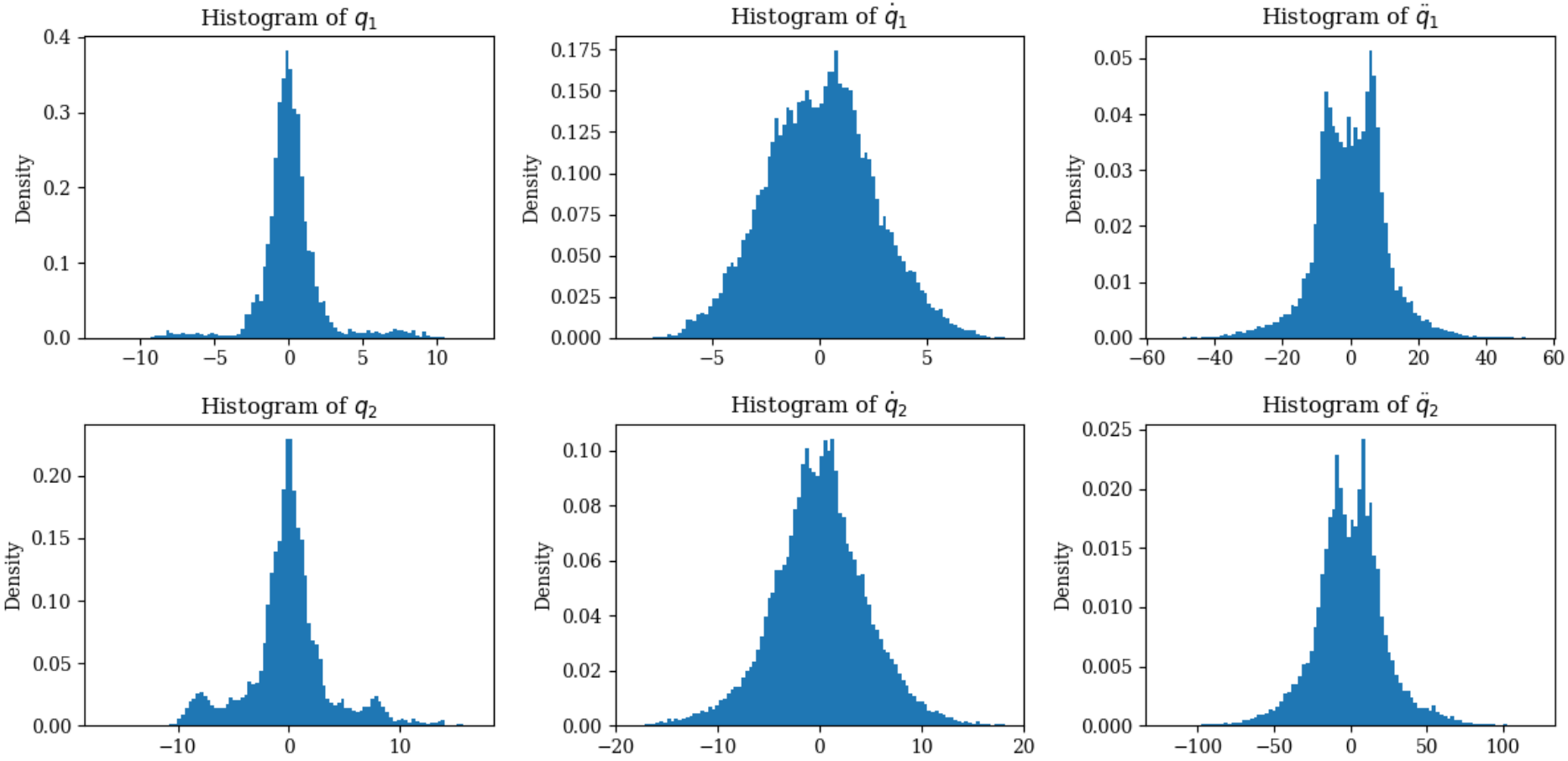
Friction

States Distribution (1)



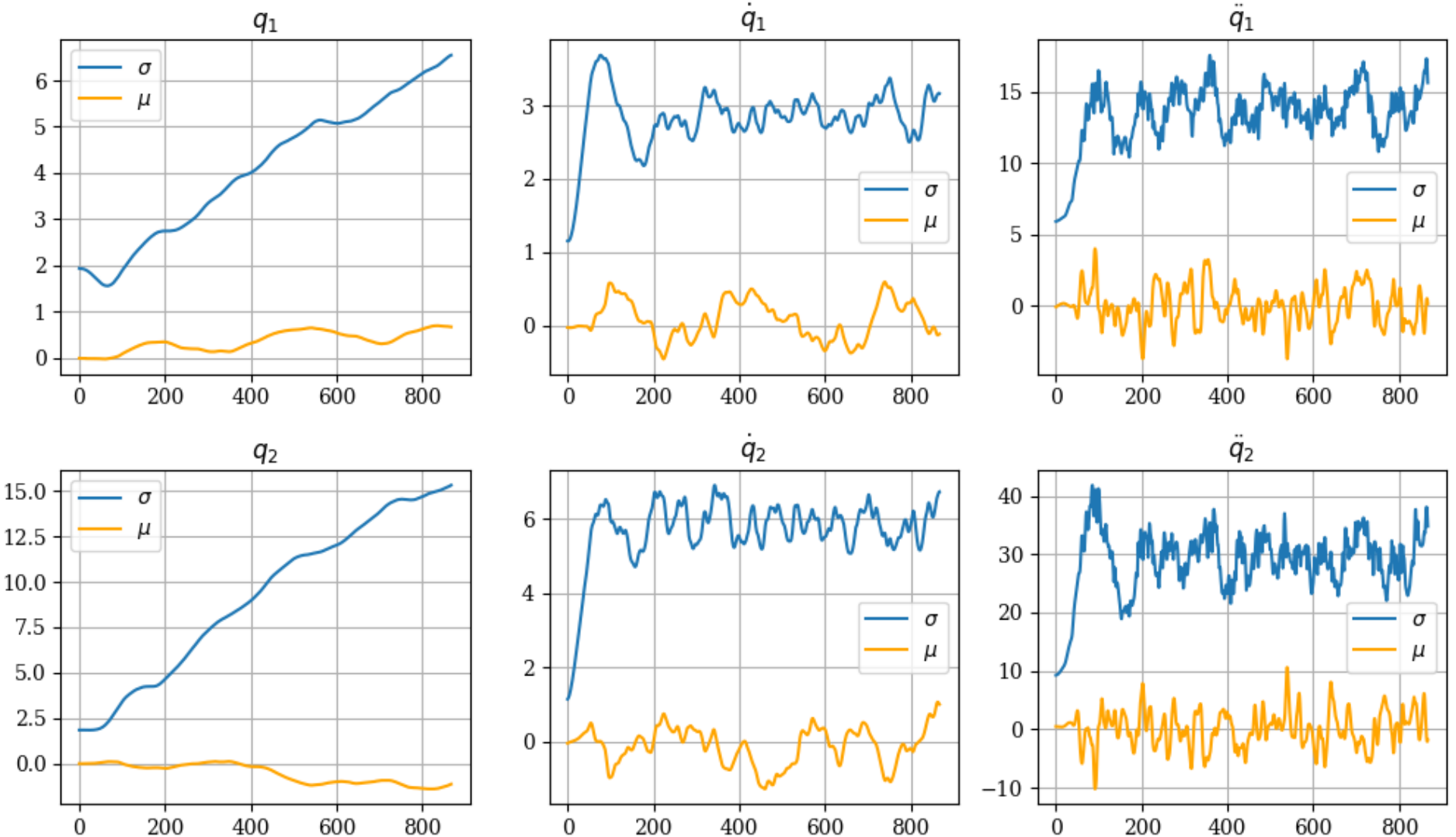
No Friction

States Distribution (2)



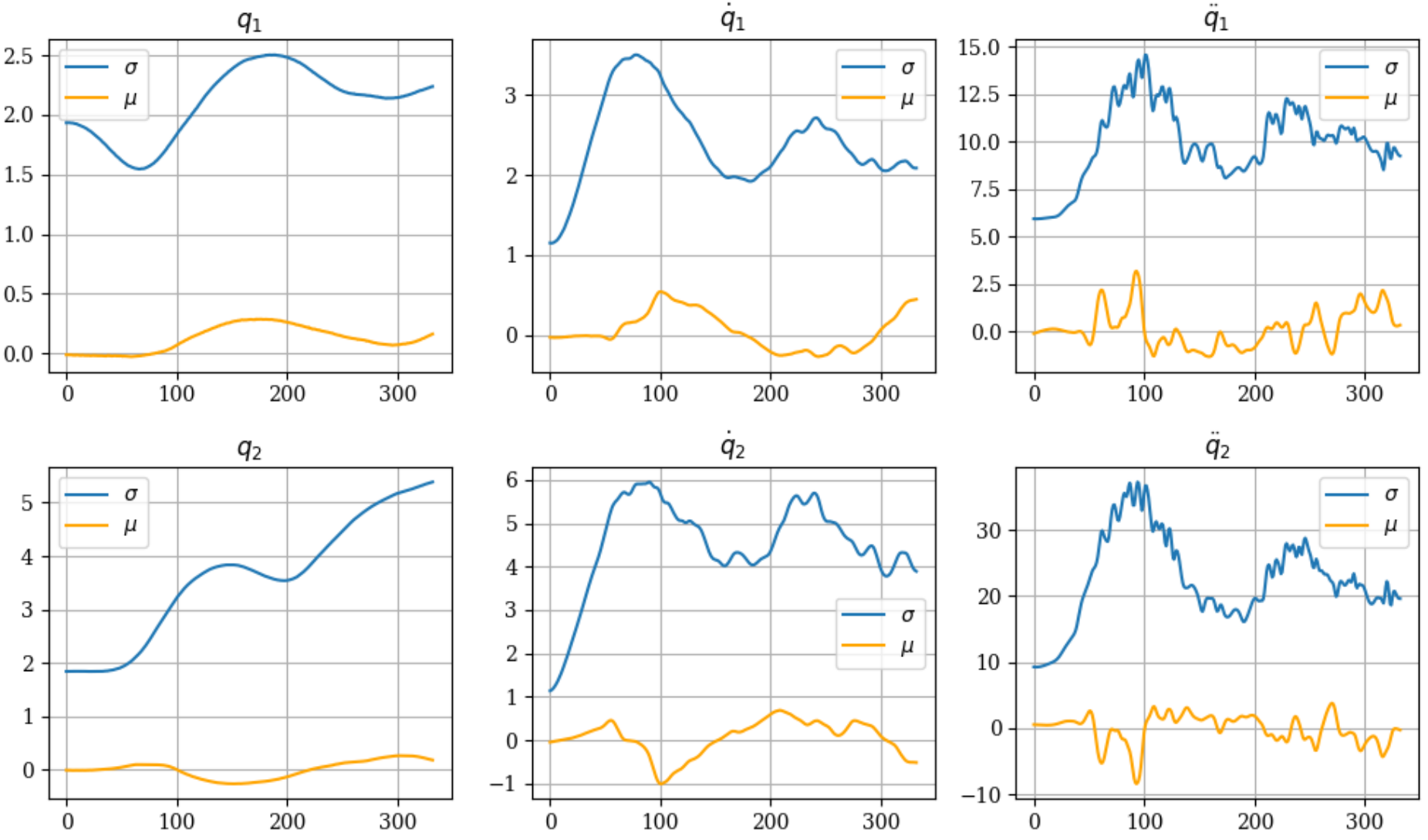
Friction

States Statistics (1)



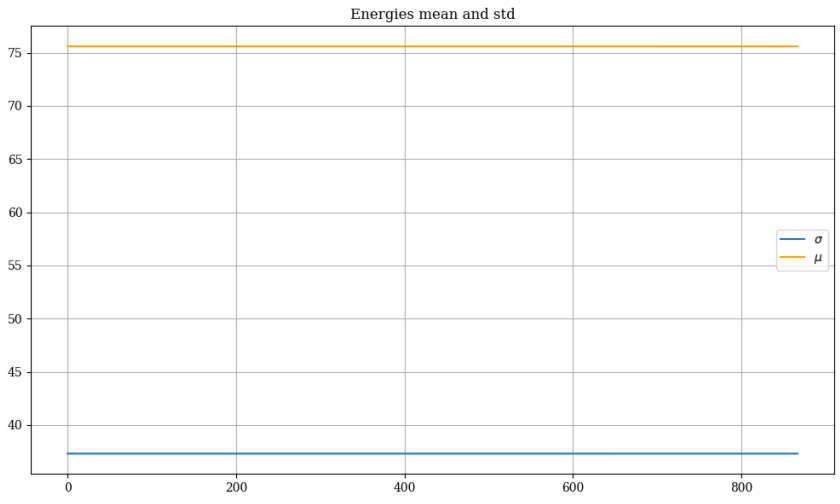
No Friction

States Statistics (2)

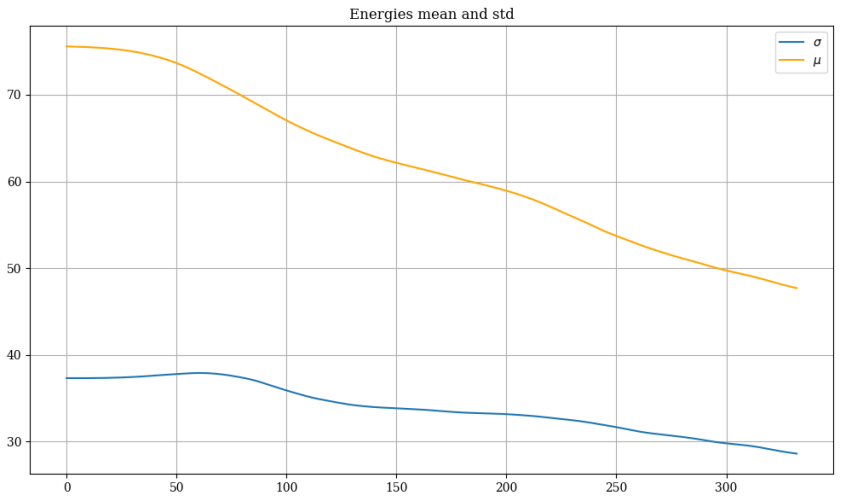


Friction

Energy Statistics



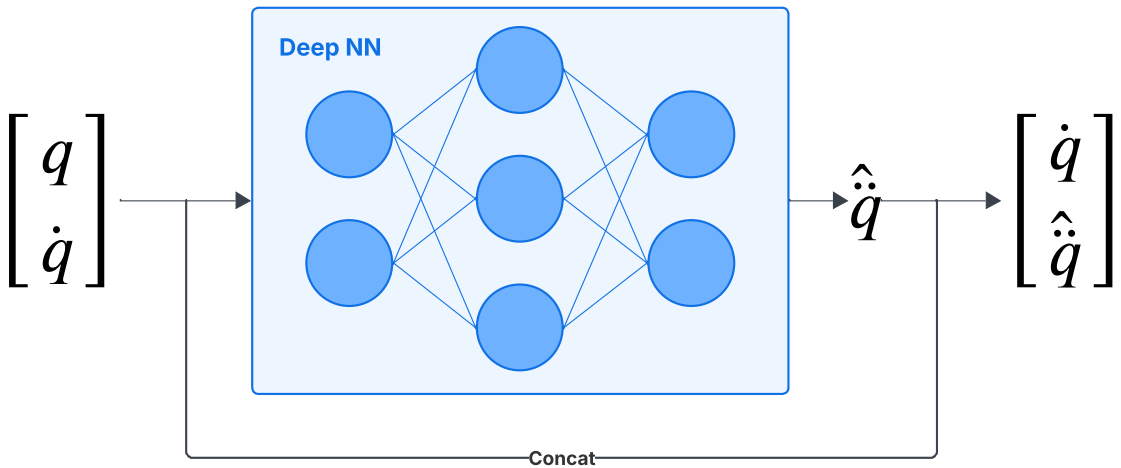
No Friction



Friction

Black Box Approaches - Architecture

Standard Neural Network for setting a **baseline** for learning the double pendulum dynamics



Training methods

- **Derivatives**: loss is computed directly on the derivatives
- **Evolution**: the derivatives are fed to a **solver** that outputs the next state. A sequential stream of predicted states is used to compute the loss

Evolution Training

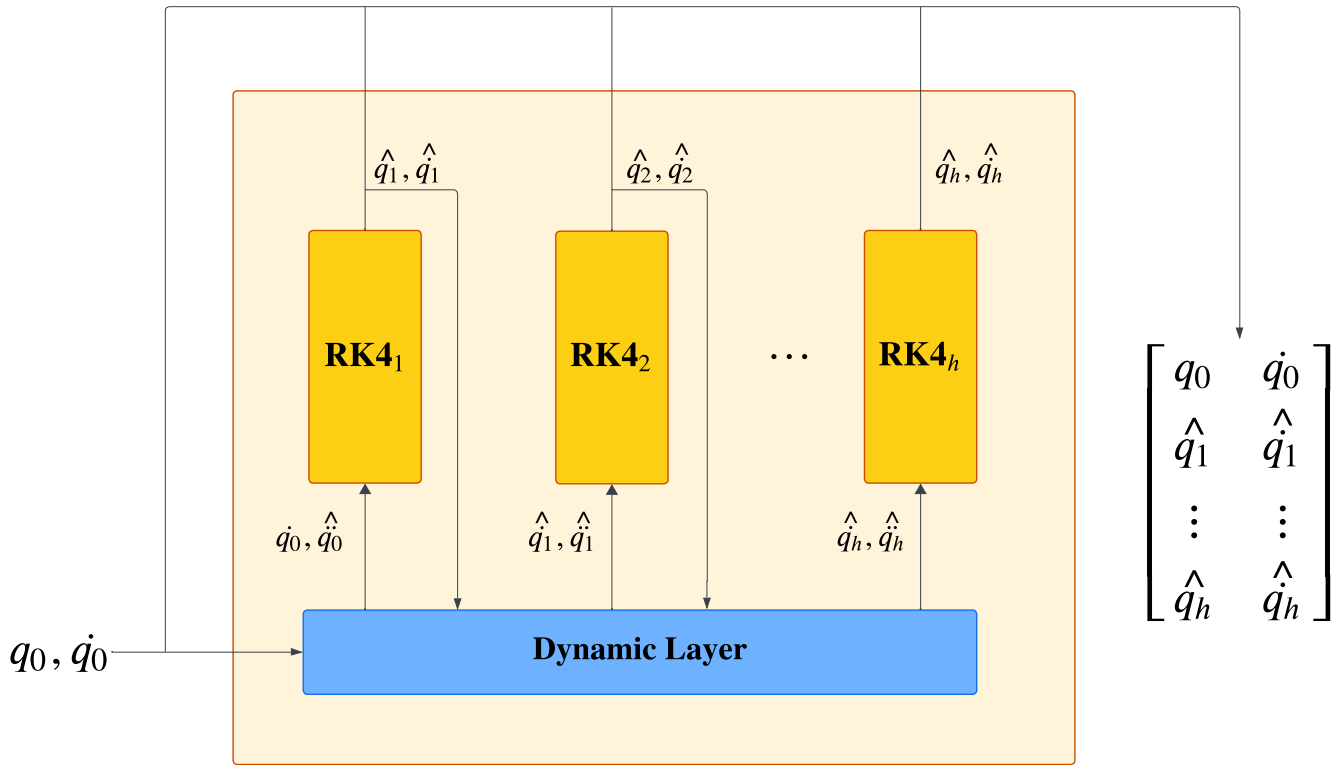


Figure: Evolution training architecture using RK4

Black Box Approaches - Training recipe (1)

Trigonometric Transformation

$$\begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \in \mathbb{R}^4 \mapsto \begin{bmatrix} \cos(q_1) \\ \sin(q_1) \\ \cos(q_2) \\ \sin(q_2) \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \in \mathbb{R}^6$$

Why do we use a TrigLayer?

- **Hard-Coding Physics (Inductive Bias):** we prevent wasting the Universal Approximator's capacity on learning basic periodic functions
- **Intrinsic Scaling:** the angular inputs are naturally bounded within the range $[-1, 1]$
- **Continuity:** the transformation ensures continuity at the angle wrap-around points (e.g., $-\pi$ to π), avoiding discontinuities that may confuse the model during training

Black Box Approaches - Training recipe (2)

Curriculum learning

Essential for **trajectory evolution** (long-term prediction)

Method: start with small horizon H, then increase progressively $H_{k+1} \leftarrow 2 \cdot H_k$

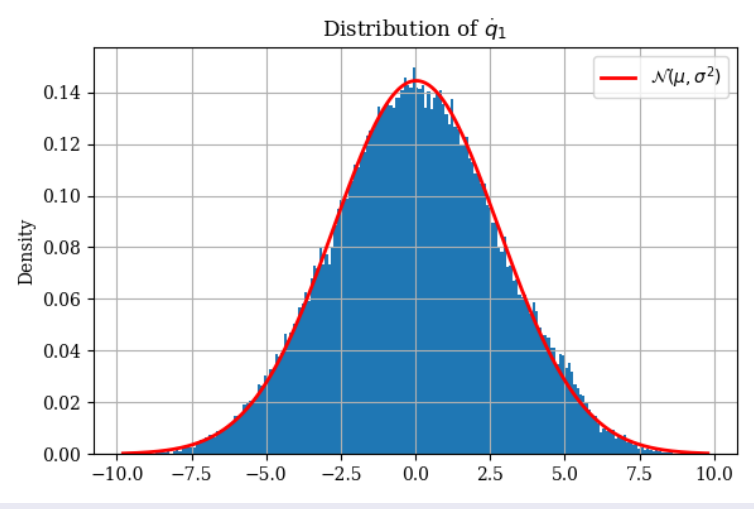
Why:

- prevents gradient explosion caused by long horizons at early training stages
- helps the model to gradually learn long-term dependencies

Black Box Approaches - Training recipe (3)

Standardization

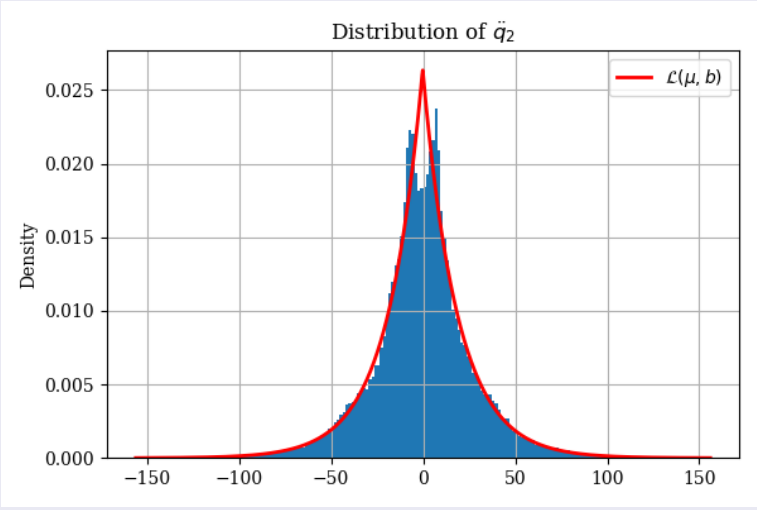
Velocities - Gaussian distribution



$$\hat{\dot{q}}_i = \frac{\dot{q}_i - \mu_{\dot{q}_i}}{\sigma_{\dot{q}_i}}$$

$$i = 1, 2$$

Accelerations - Laplacian distribution



$$\hat{\ddot{q}}_i = \frac{\ddot{q}_i - \mu_{\ddot{q}_i}}{\sqrt{2b}}$$

with $b = \mathbb{E}\left[|q - \tilde{\mu}|\right]$

Black Box Approaches - Training recipe (4)

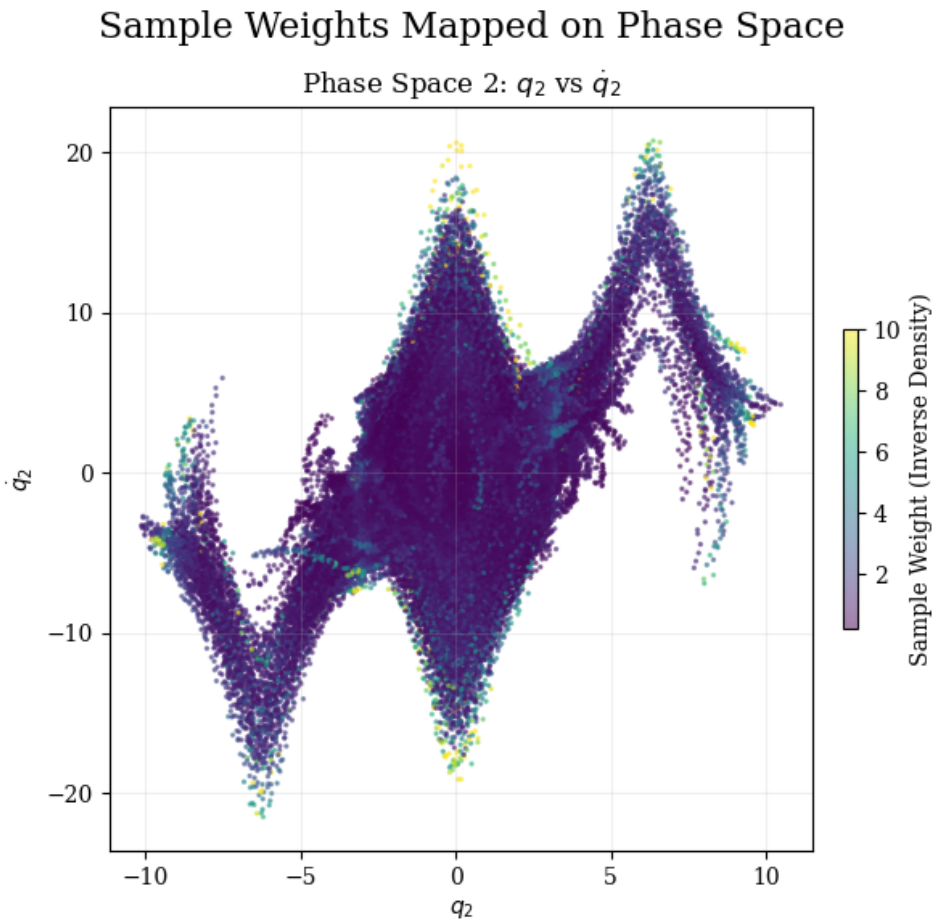
Sample weights

Computed using the 4D KDE of the input distribution. In particular:

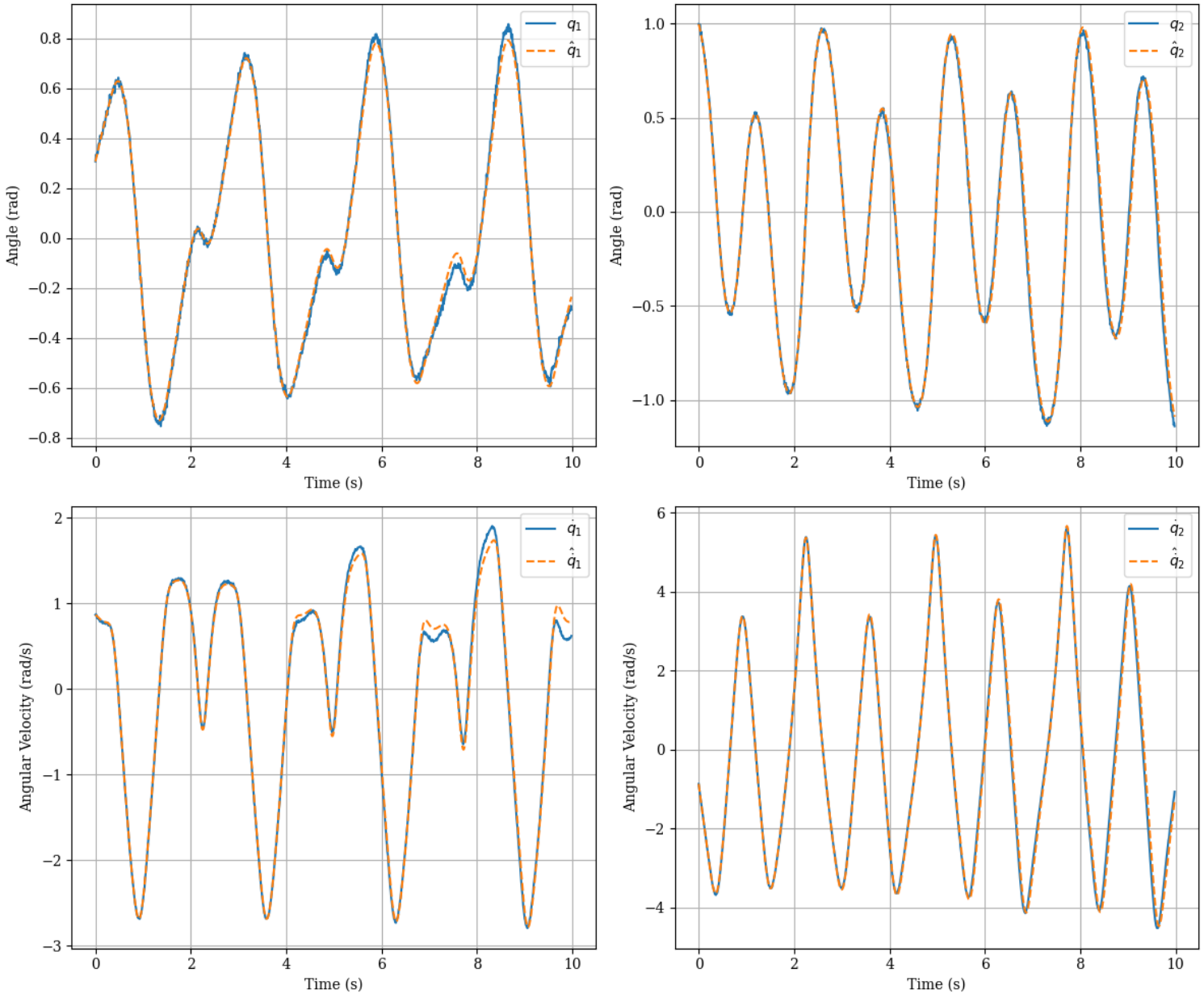
$$\sigma(q_1, q_2, \dot{q}_1, \dot{q}_2) = \\ = \frac{h_{tot}}{h(q_1, q_2, \dot{q}_1, \dot{q}_2)}$$

then normalized such that $\sum \sigma_i = 1$

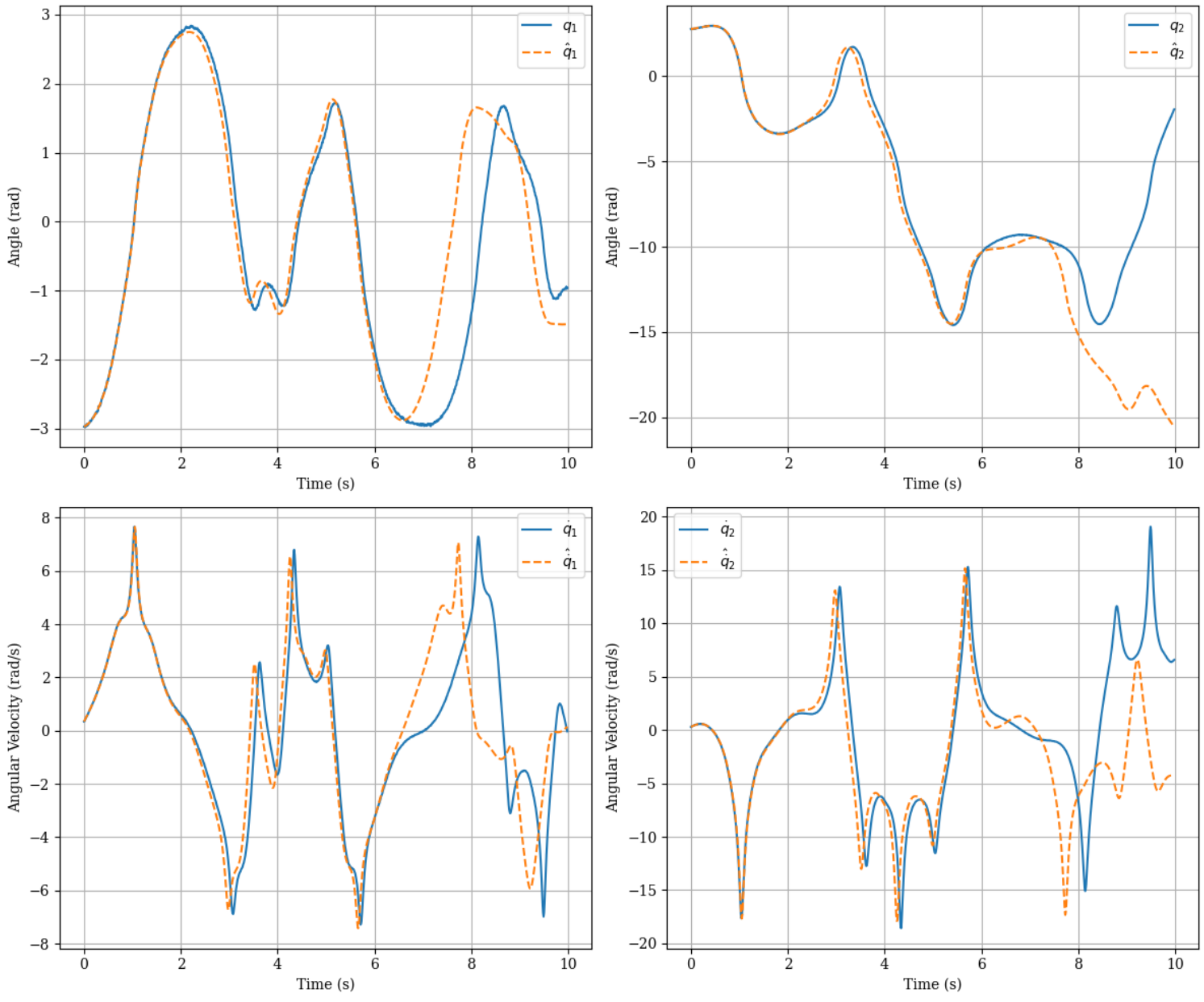
Important: multivariate distribution capture any **coupling term** in the dynamics



Black Box Approaches - Results (1)

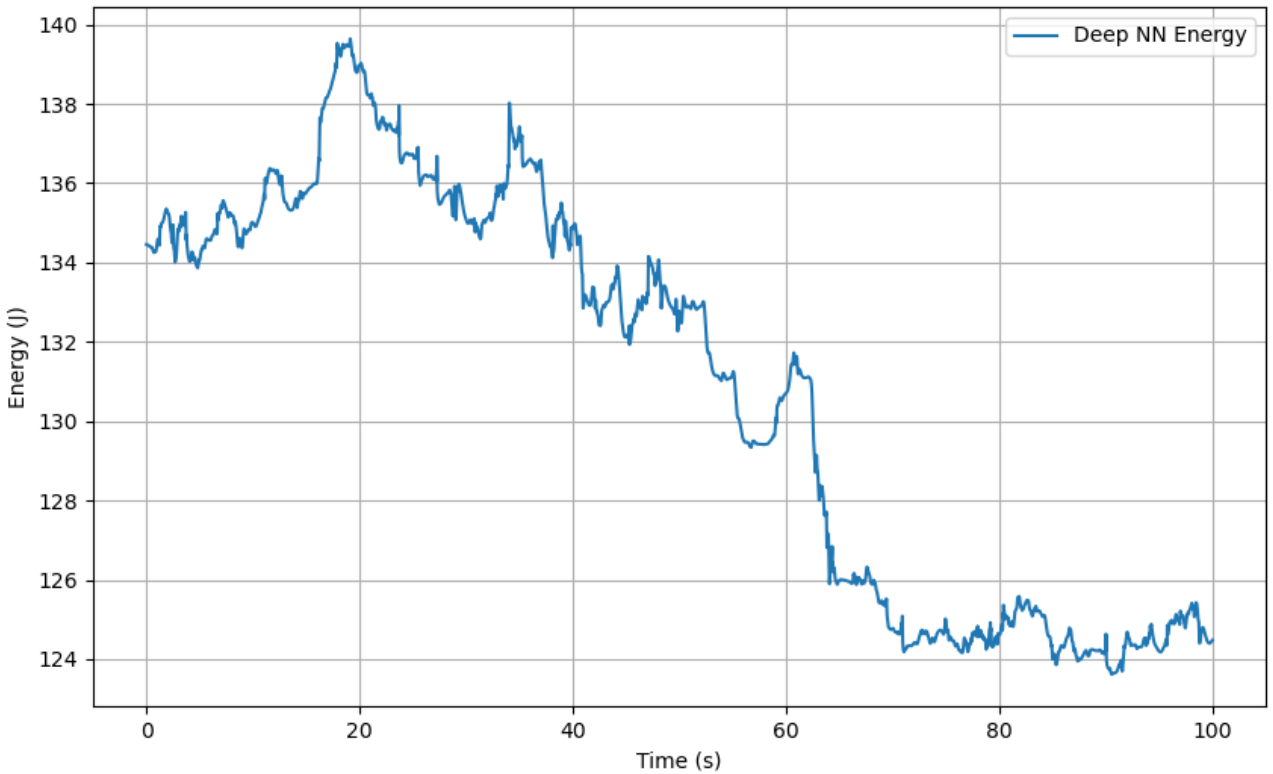


Black Box Approaches - Results (2)



Black Box Approaches - Energy drift

Energy evolution over time with **no friction**



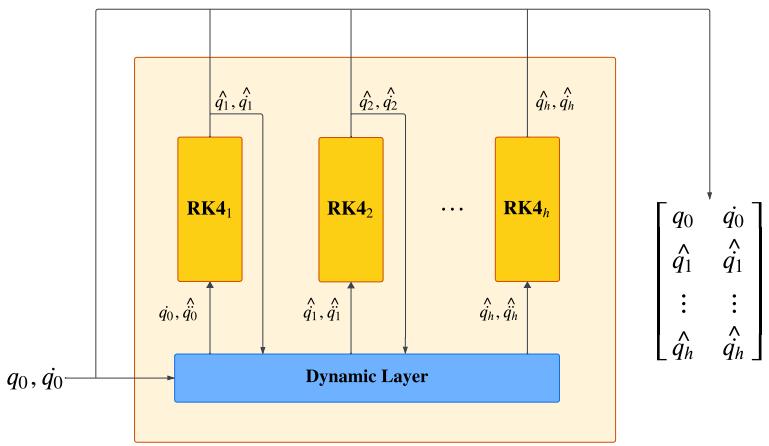
The system is **not able to conserve energy**.

White Box Approach

Physical Parameters Estimation

- **Assumptions:** physical parameters are fixed over time (time-invariance)
- **Knowledge:**
  - Analytical dynamics
  - System state trajectories $(q, \dot{q})$
- **Goal:** estimate the unknown parameters

$$m_1, m_2, l_1, l_2, b_1, b_2$$



Physical Parameters Estimation - Architecture

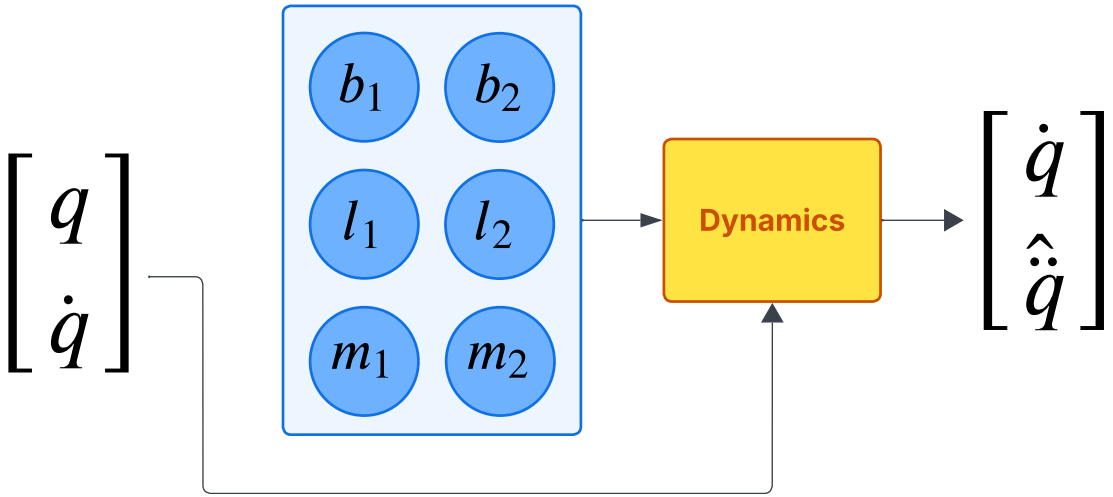


Figure: ODE Architecture

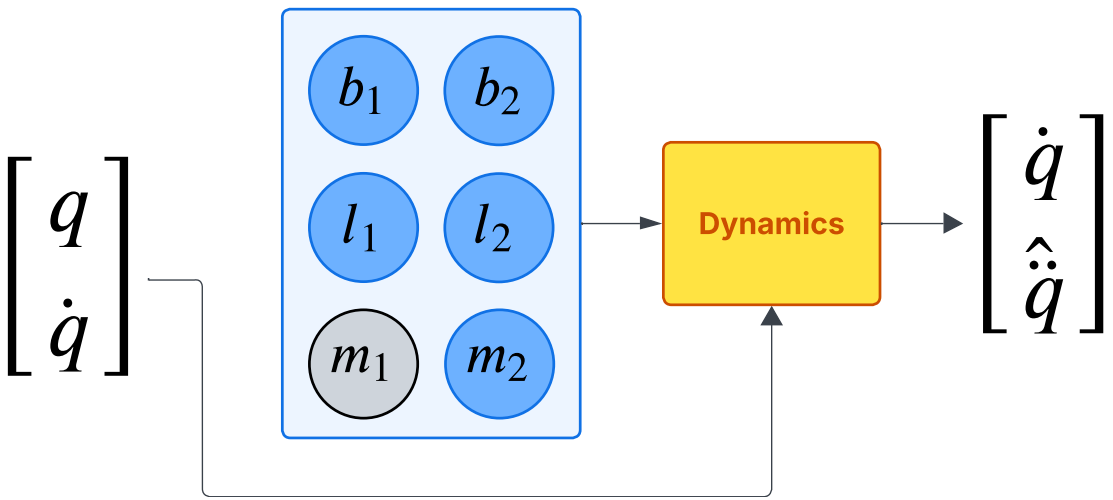


Figure: ODE with Fixed Parameter

Physical Parameters Estimation - Results

Observations:

- **Gauge Invariance Property**
- Convergence
- **Correct Evolution** $\rightarrow$ deviations over long time.
- **Energy evolution**: matching (*by-construction*).
- low **Max Parameter Discrepancy**

Evolution Testing

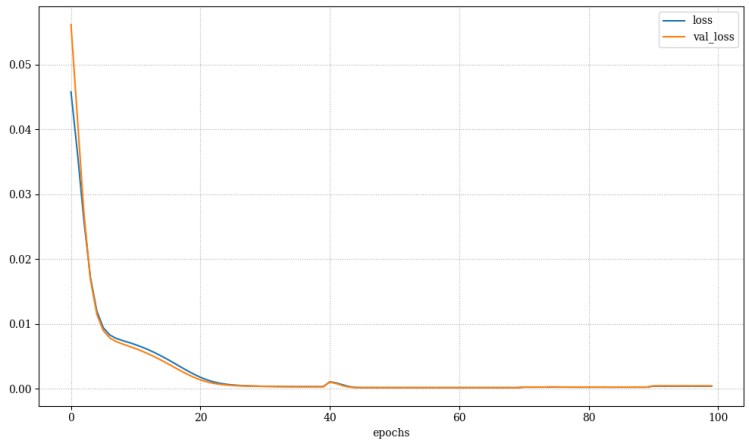


Figure: Loss History

Parameter	Estimated	true
m_1	2.200	2.20
m_2	1.799	1.800
l_1	1.199	1.200
l_2	1.000	1.000
b_1	0.295	0.300
b_2	0.300	0.300

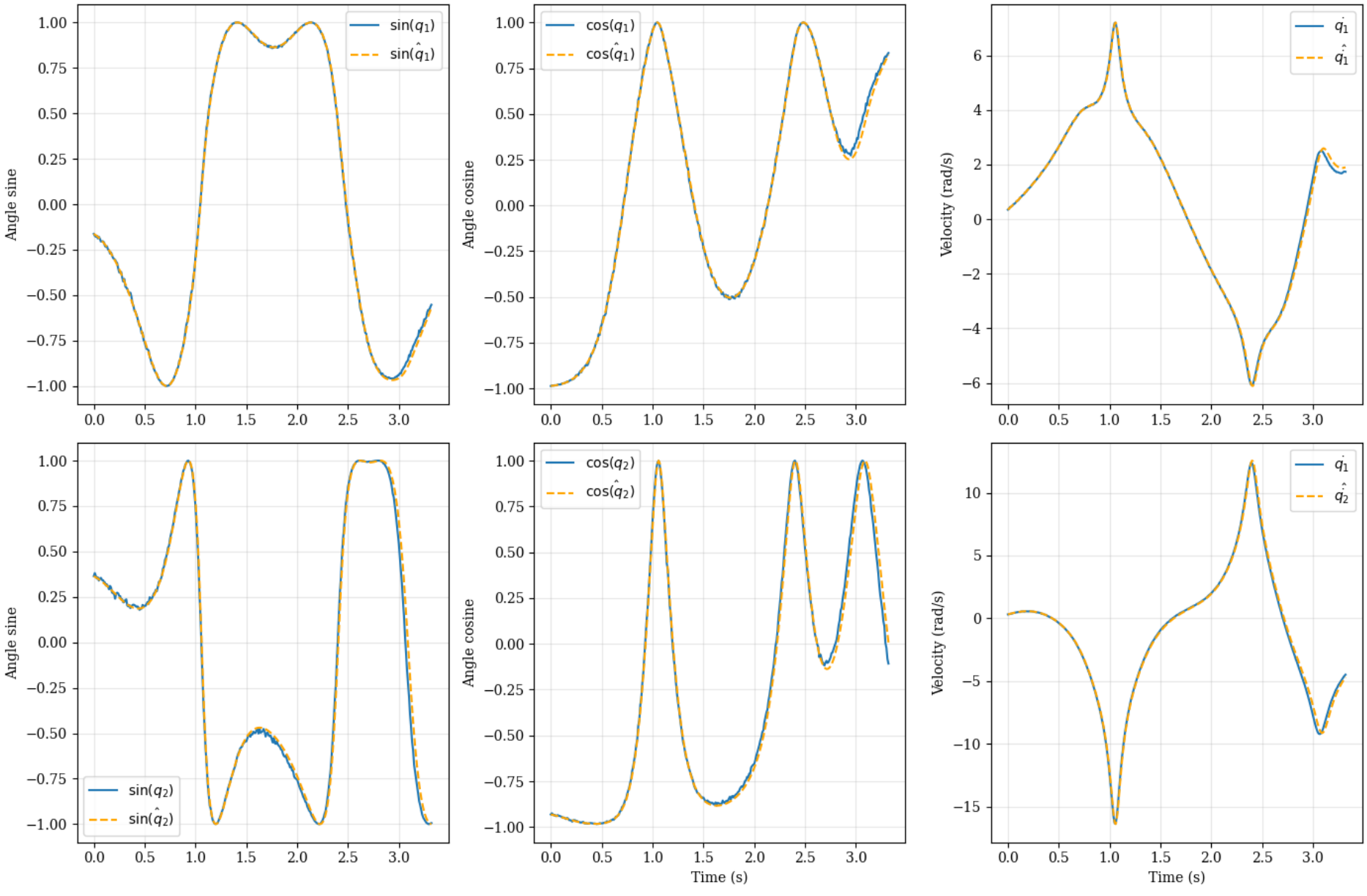


Figure: Normal Evolution

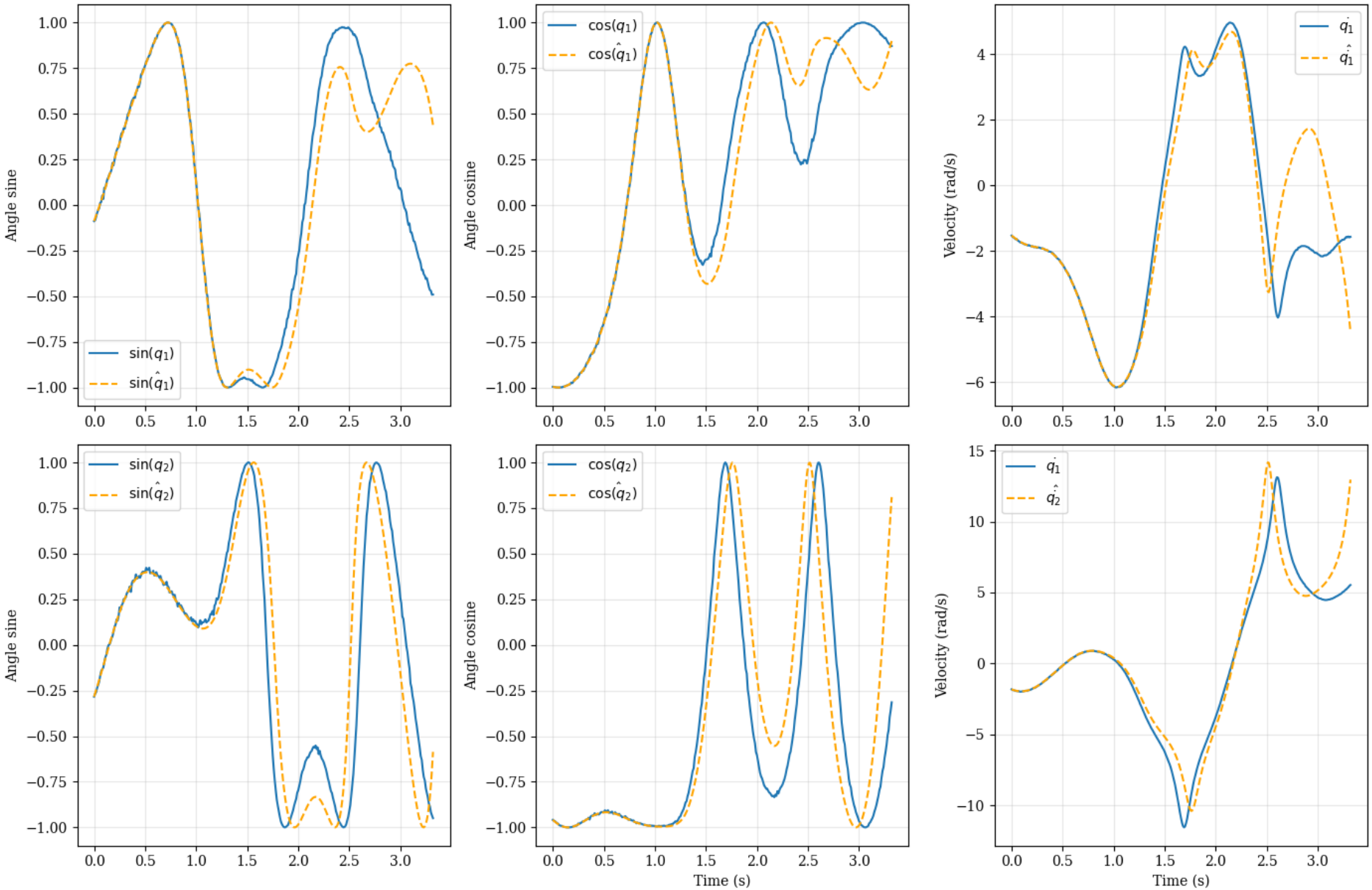


Figure: Hard Evolution

Hamiltonian Neural Networks (HNN)

Choosing the Hamiltonian would be the **best option** but a set of **canonical variables** is needed.

Canonical Variables

$$\mathbf{z} = \begin{bmatrix} \mathbf{q} \\ \mathbf{p} \end{bmatrix}$$

- **q**: generalized displacements
- **p**: generalized momenta

Given the **Hamiltonian** as $\mathcal{H}$, it's possible to find the dynamics with:

$$\frac{dq}{dt} = \frac{d\mathcal{H}}{dp} \qquad \frac{dp}{dt} = -\frac{d\mathcal{H}}{dq}$$

It's possible to use an MLP neural network that learns the Hamiltonian $\mathcal{H}_\theta$ and a loss that enforces the dynamics:

$$\mathcal{L}_{HNN} = \left\| \frac{d\mathcal{H}_\theta}{dp} - \frac{\partial q}{\partial t} \right\|_2 + \left\| \frac{d\mathcal{H}_\theta}{dq} + \frac{\partial p}{\partial t} \right\|_2$$

Two problems of momenta:

- they depend on the parameters of the system
- more importantly they depend on the coupled dynamics not known

Entries Prediction

Remembering equation (4), is it necessary to know matrices structure thoroughly?

Matrices Structure

Goal $\rightarrow$ **Graybox** approach + insights on system properties.
GT (Ground Truth) $\rightarrow$ **Newton-Euler** algorithms.
Application: **Quick inference** for control purposes.

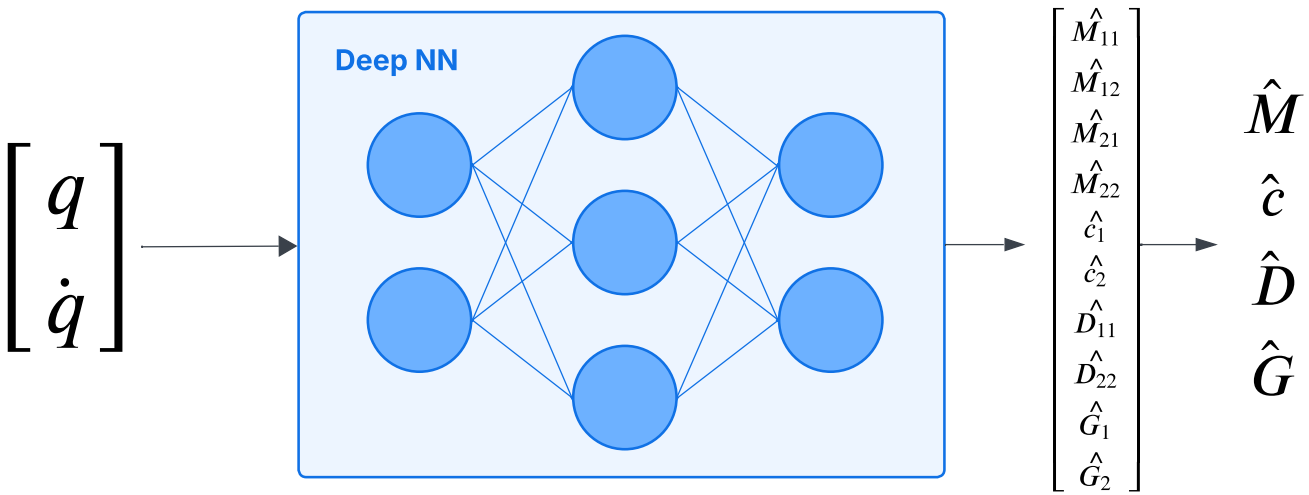
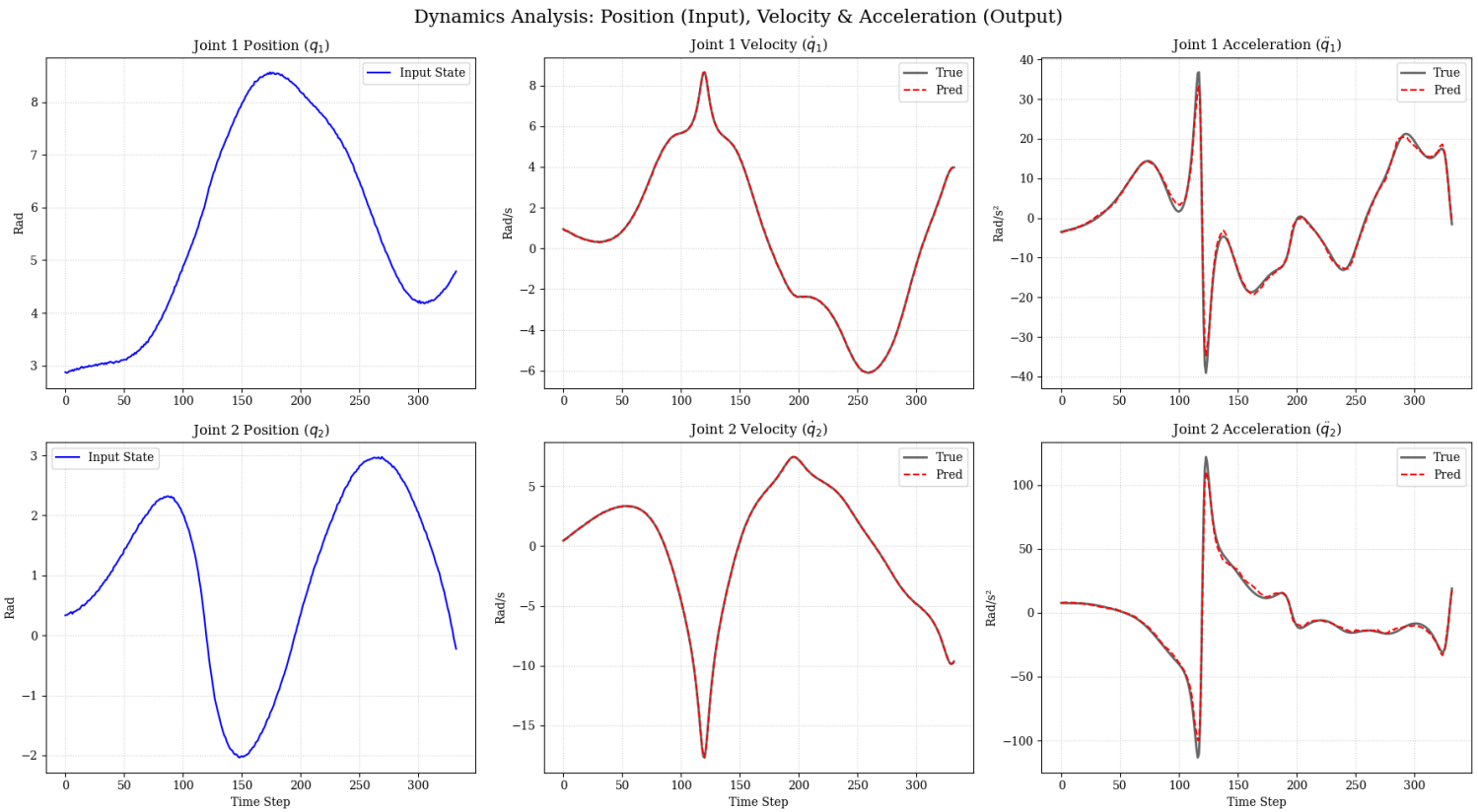
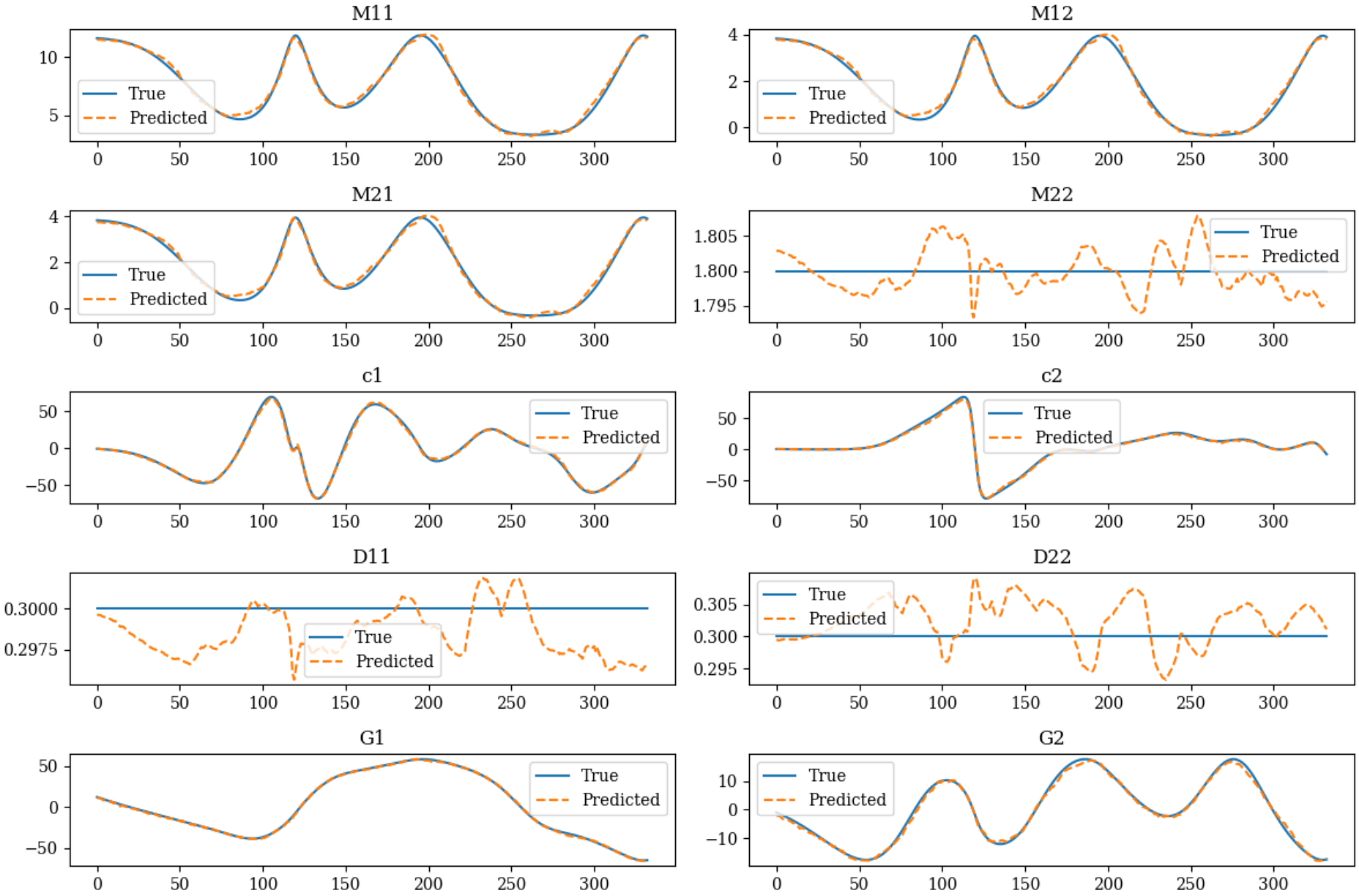


Figure: Entries prediction architecture

Entries Prediction - Considerations

- **Advantages**: Quick training, very good results.
- **Disadvantages**: Requires GT values for matrices entries.
- We would like to obtain the same results **without** knowing the entries.

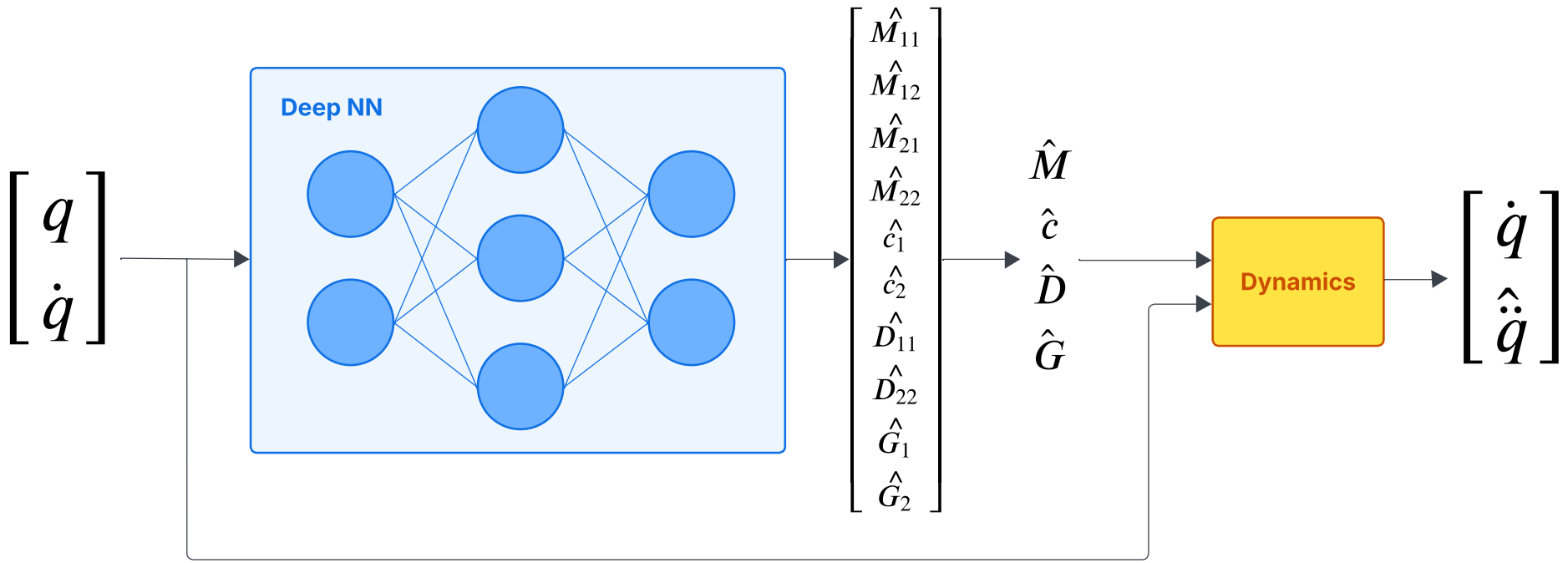



M11
M12
M21
M22
c1
c2
D11
D22
G1
G2
True
Predicted

UDE – Uninformed

Idea

Universal Differential Equations.
Model Layers: Can we exploit Evolution Learning architecture.



UDE – Uninformed - Considerations

Matrix Inversion

The equations of motion can be rewritten as:

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}[\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{D}\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q})]$$

- Much time to converge: RK4 integration combined with deep backpropagation.
- Not constrained : no physical knowledge, not even good sense.
- **Initial Results**: Often **poor results** due to instability.
- **Ill-Posing**: Many **degrees of indeterminacy**.

Lagrangian Neural Networks (LNN) - Architecture

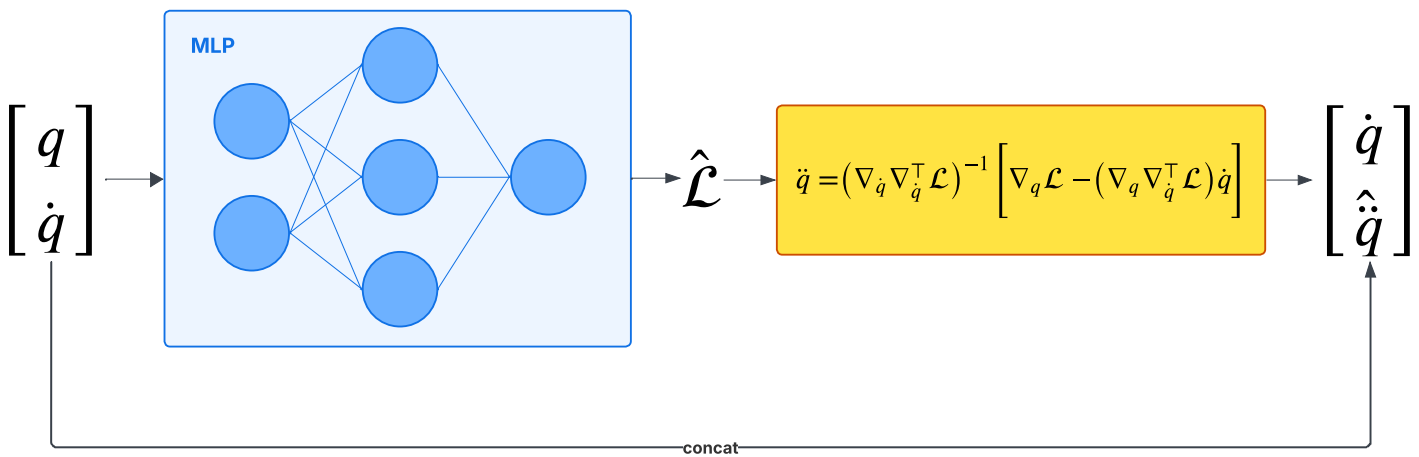
Euler-Lagrange equation can be **discretized** and expanded with chain rule

$$\frac{d}{dt}\nabla_{\dot{q}}\mathcal{L} - \nabla_q\mathcal{L} = 0 \rightarrow \left(\nabla_{\dot{q}}\nabla_{\dot{q}}^{\top}\mathcal{L}\right)\ddot{q} + \left(\nabla_q\nabla_{\dot{q}}^{\top}\mathcal{L}\right)\dot{q} = \nabla_q\mathcal{L}$$

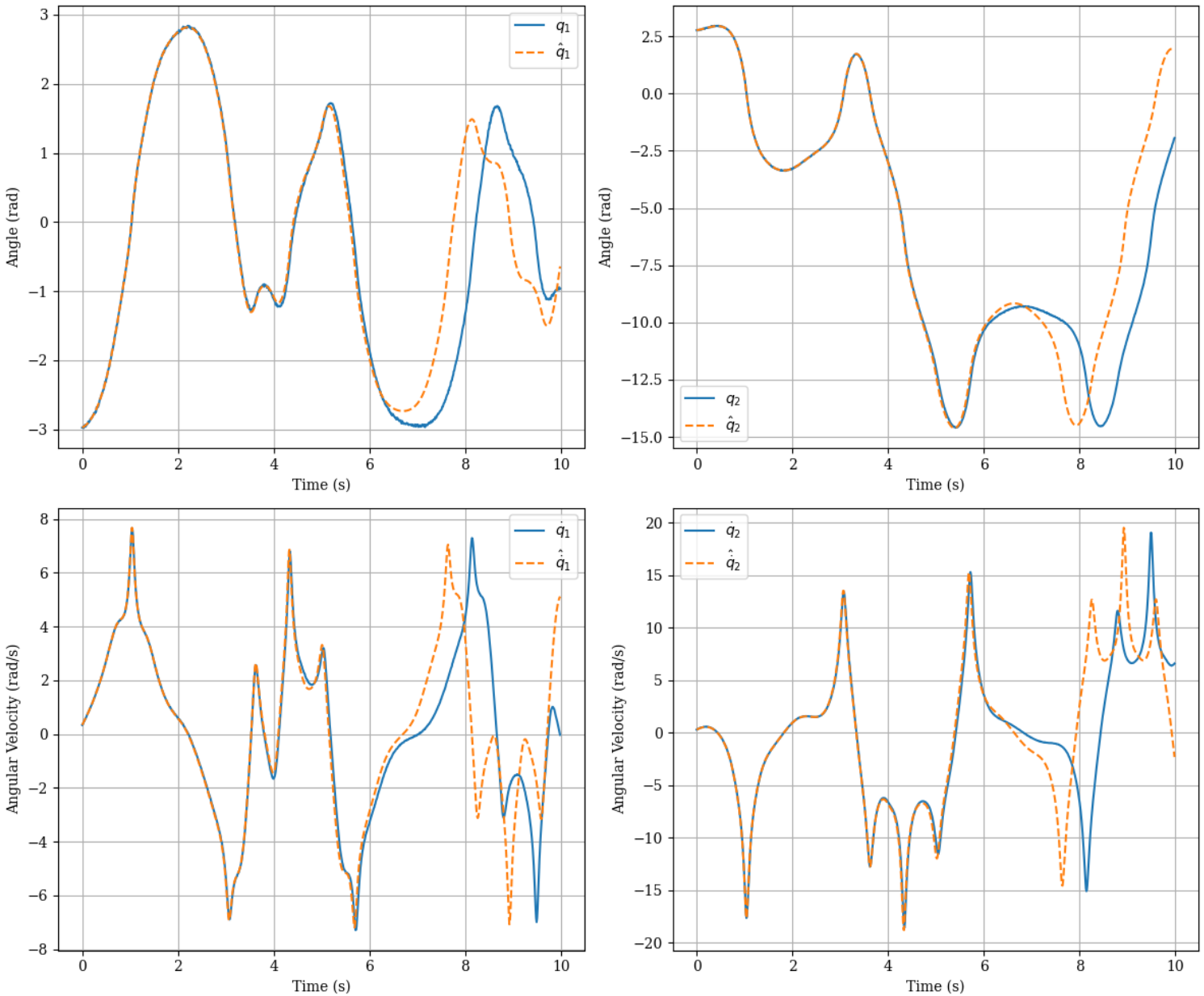
Since the hessian $\nabla_{\dot{q}}\nabla_{\dot{q}}^{\top}\mathcal{L}$, which is the **Inertia matrix**, is positive definite:

$$\ddot{q} = \left(\nabla_{\dot{q}}\nabla_{\dot{q}}^{\top}\mathcal{L}\right)^{-1}\left[\nabla_q\mathcal{L} - \left(\nabla_q\nabla_{\dot{q}}^{\top}\mathcal{L}\right)\dot{q}\right]$$

$\mathcal{L}$ is the output of a black box neural network, while the Lagrangian Equation is computed through internal chains of gradients in the network

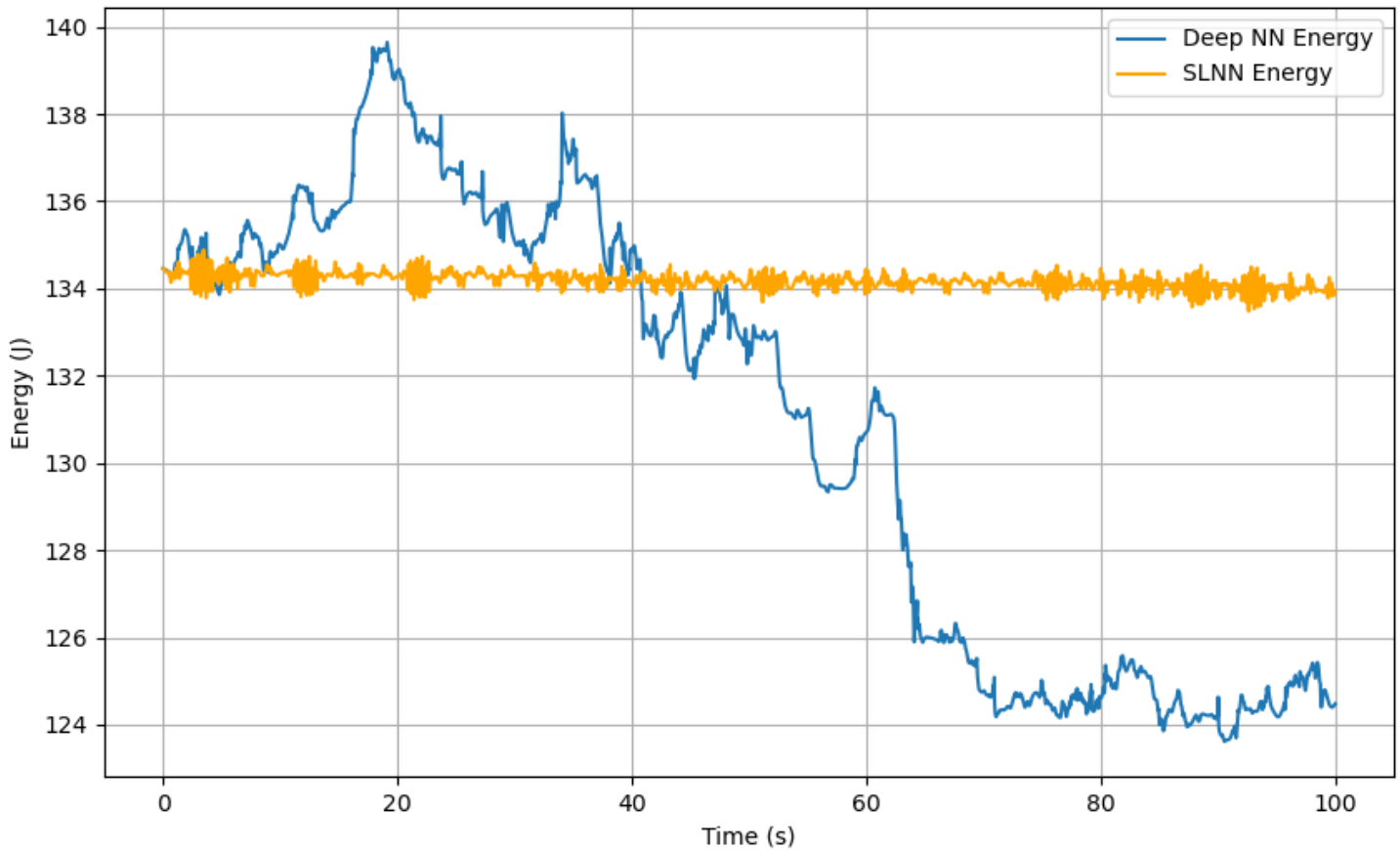


Lagrangian Neural Networks (LNN) - Evolution



Lagrangian Neural Networks (LNN) - Considerations (1)

- Almost perfect **Energy** conservation



Lagrangian Neural Networks (LNN) - Considerations (2)

- positive definiteness of **Hessian** is not guaranteed being $\mathcal{L}$ black box, causing **instability**
- **Batch Size**: Very small $\rightarrow$ need for **noisy gradients** to escape shallow loss regions (as suggested in literature).
- **Speed Issues**: Hessian computation is expensive.
- **Anomaly**: Difficulty learning **non-conservative systems**, including dynamic models with mistakes.

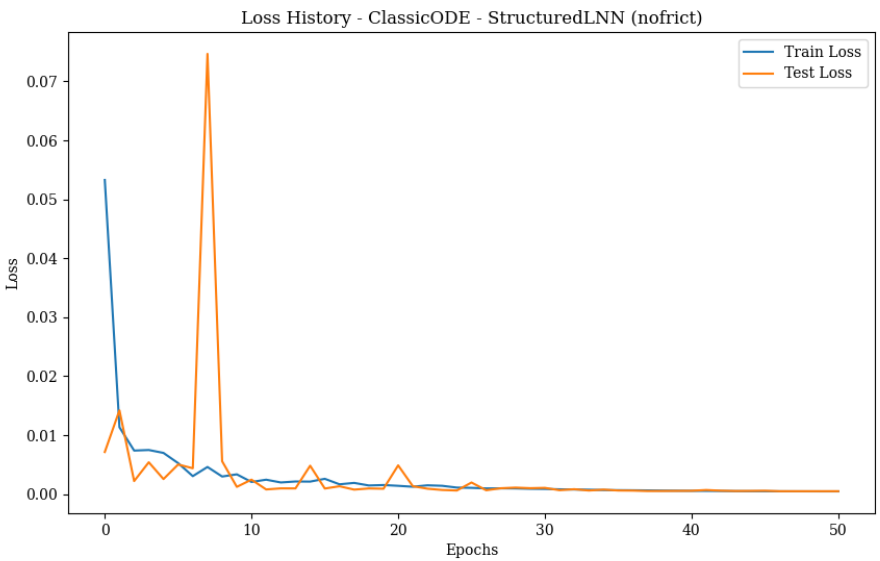


Figure: loss descent - conservative

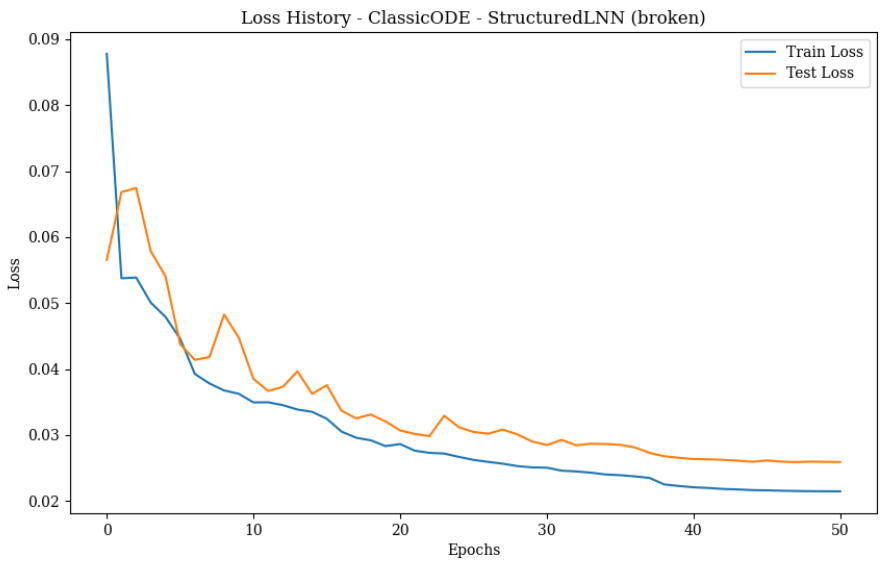
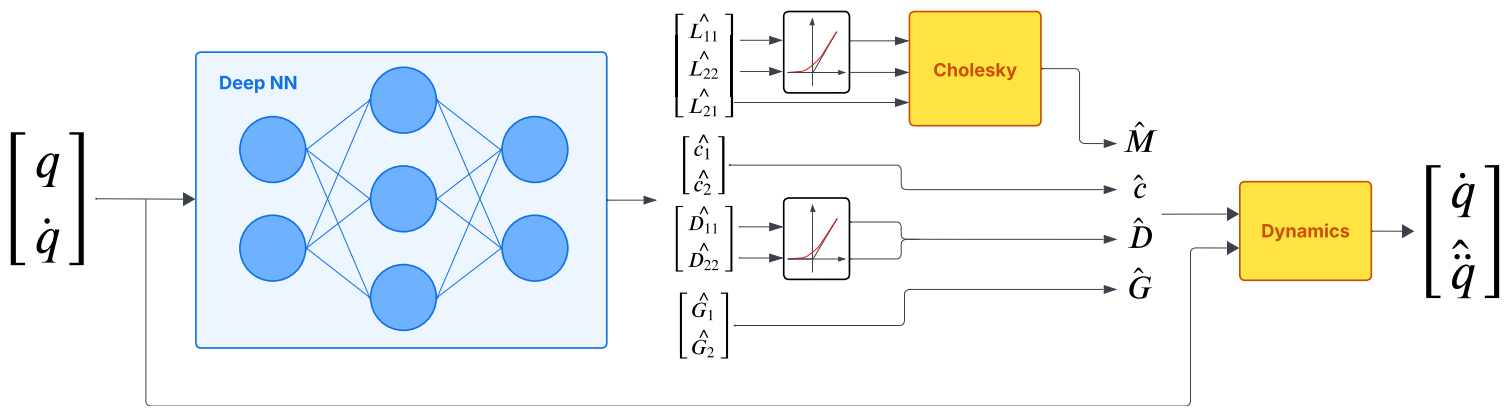


Figure: loss descent - non-conservative

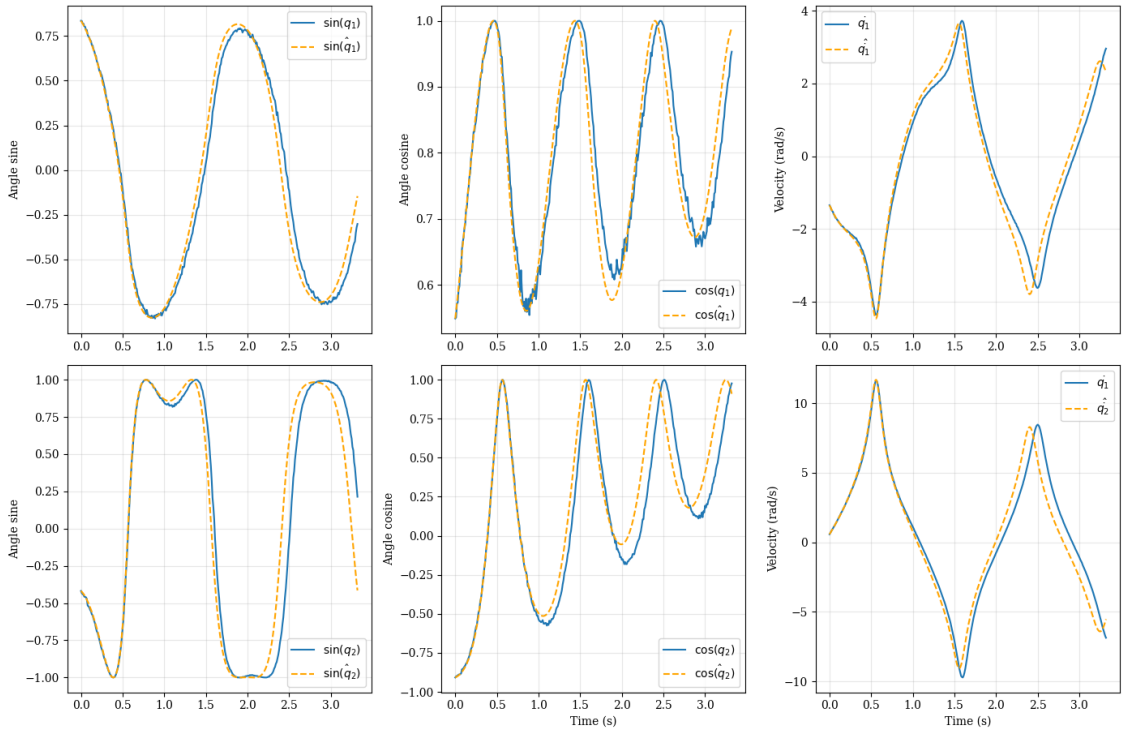
Informed UDE

Exploiting All Physical Knowledge:

- Inertia matrix **M**: **Positive definite** and **symmetric**.
- Friction: **Positive definite**, **diagonal/constant** matrix.
- Last term in $\mathbf{M(q)}$ matrix is constant as well as $\mathbf{d_{11}}$ and $\mathbf{d_{22}}$ in **D** which is diagonal.
- **Implementation**: Use of **Cholesky decomposition** to guarantee **M** positive definite.



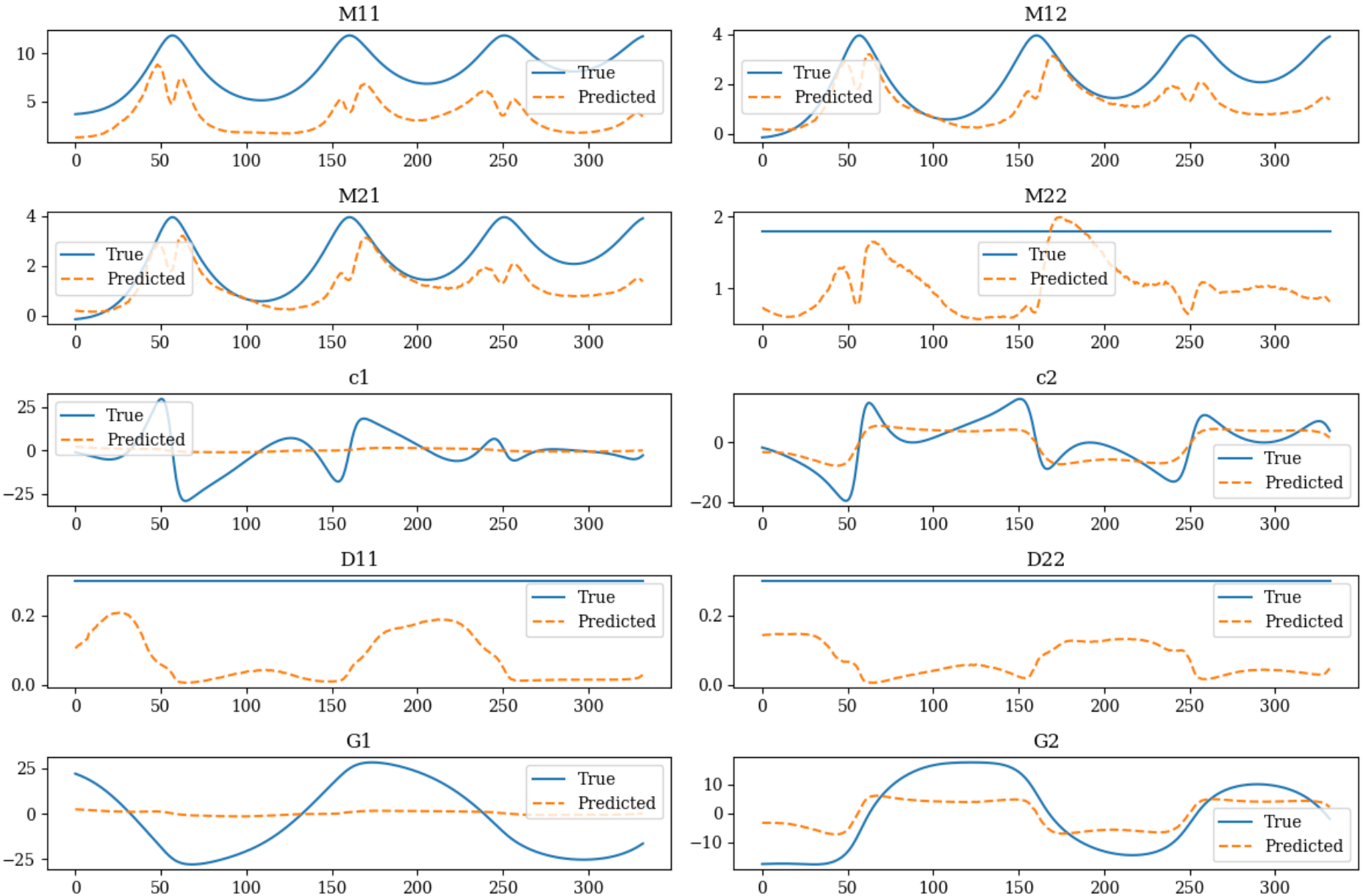
Informed UDE - Results



- **Implementation**: Parameters now make **physical sense** for sure.
- Takes less time to converge
- the system is **stable**

Question

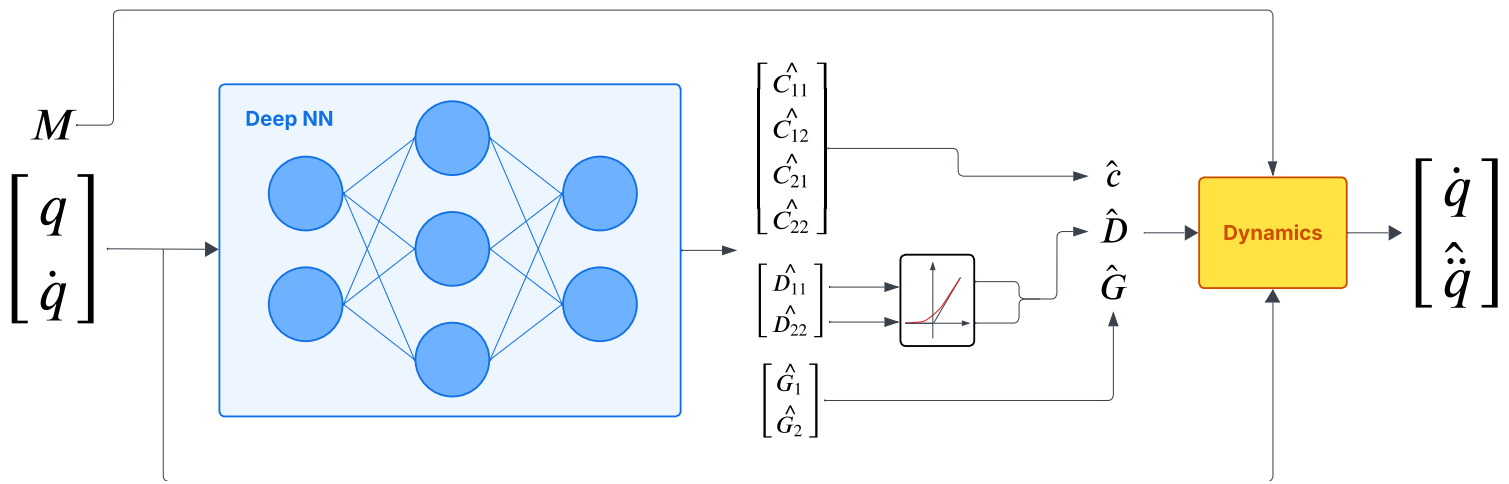
Can we plug in even more knowledge?


M11
M12
M21
M22
c1
c2
D11
D22
G1
G2
True
Predicted

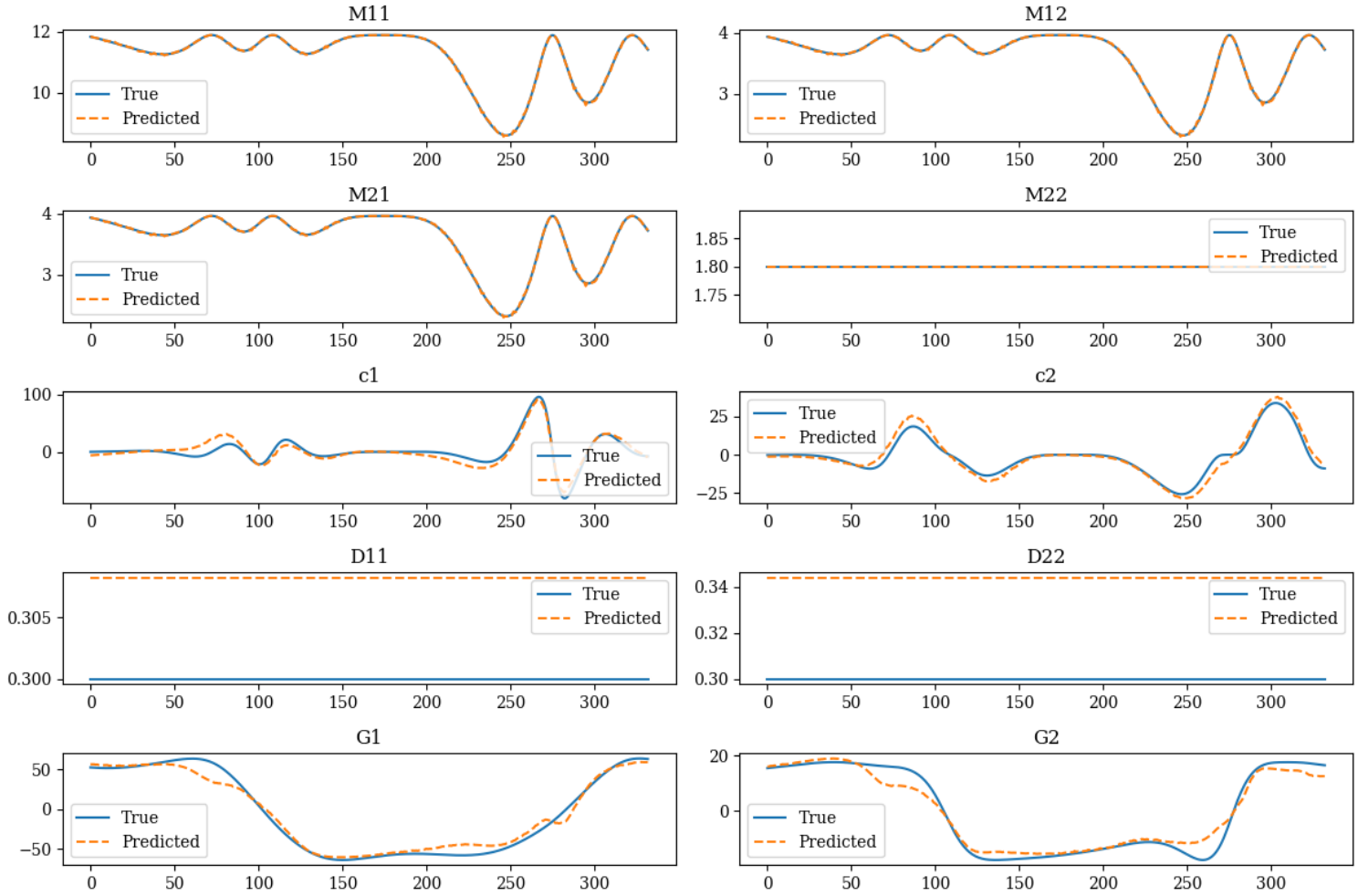
"More" Informed UDE

Idea

If we somehow could give the matrix M to the system, we would give an anchor to the learning.

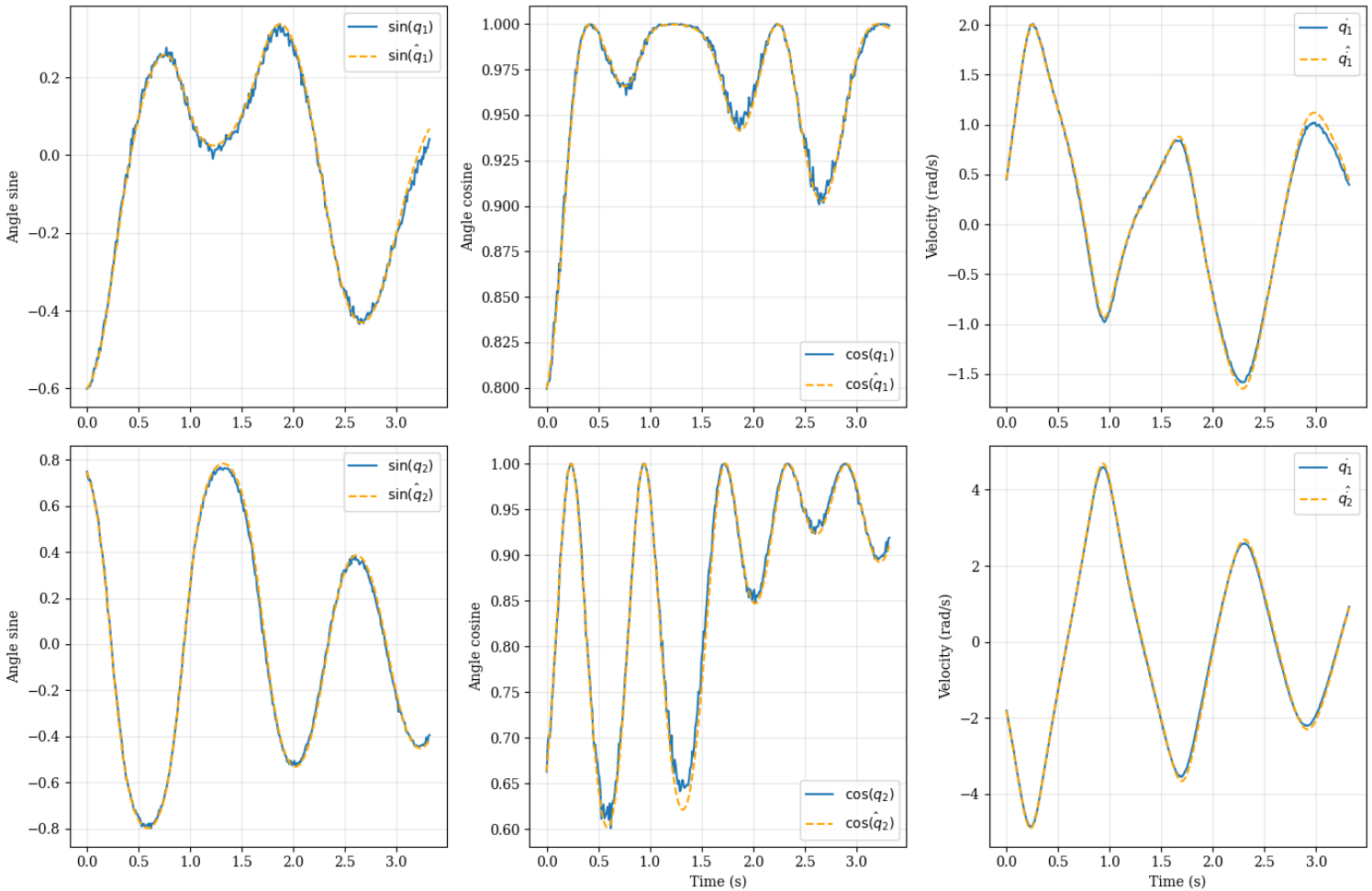


"More" Informed UDE - Results



- **Easier** identification of entries.
- Somehow more determinate: less degrees of freedom.

"More" Informed UDE - Evolution



Informed UDE - Drawing Conclusions

- Almost perfect **identification** of physical parameters.
- Informed UDE and Phase Portraits.
- Brute force controllers vs Adaptive controllers.
- We could do even better : $\dot{M}(q) - 2C(q, \dot{q})$ should be **skew-symmetric**.

Table: Comparison of the three UDE methods

Method	**Time**	**Informations**	**Comments**
Uninformed	High	None	Unstable, Dumb
Informed	Medium	Structural	Stable, Gauge
M. Informed	Medium	M matrix	Identifiable, Difficult

Metric

Maximum Parameter Difference: respectively not assigned, 24.02, 6.94.

Structured Lagrangian Neural Network (SLNN) - Architecture (1)

Instead of learning a black box $\mathcal{L}$ it is possible to expolit known physical properties of mechanical systems.

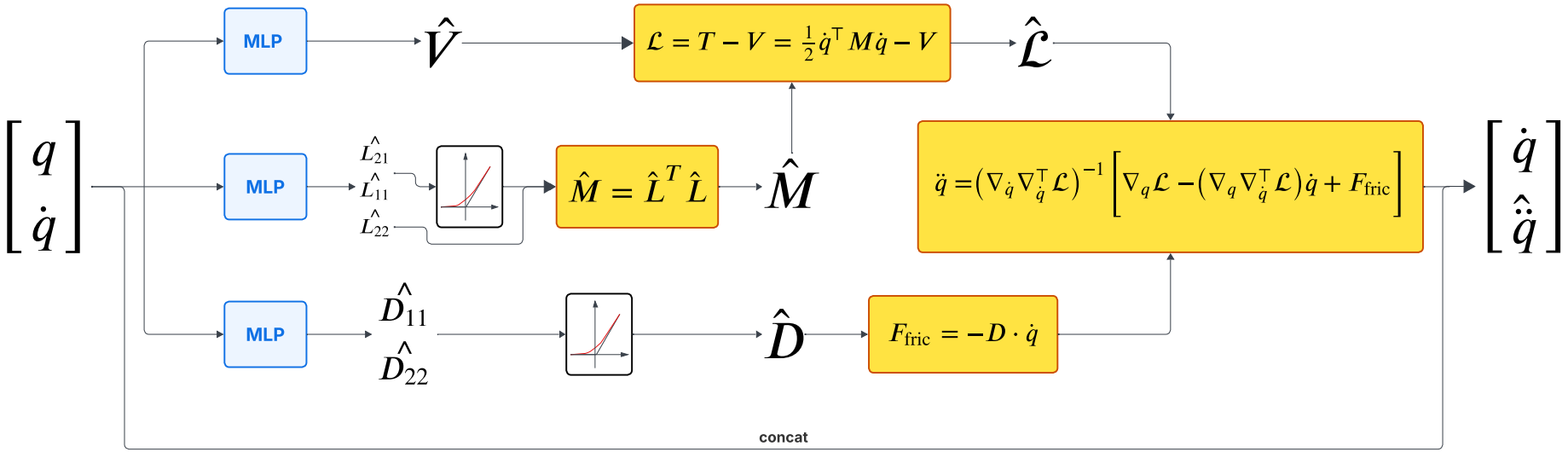
$$\mathcal{L} = T - V = \frac{1}{2}\dot{q}^\top M\dot{q} - V$$

- To learn a positive definite M it is possible to use **Cholesky decomposition**:

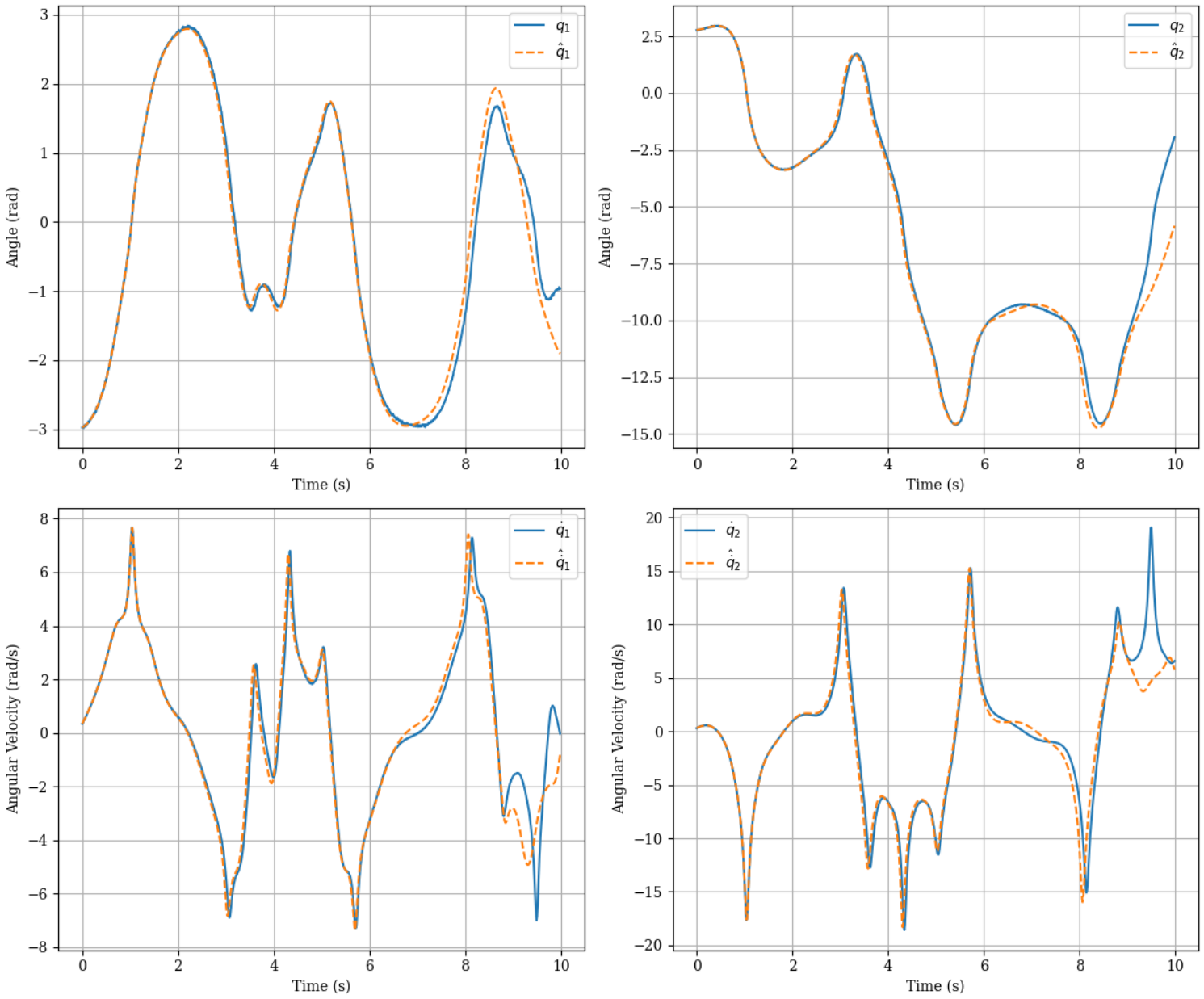
  $$M = LL^\top$$

  Where L is a learned **lower triangular** matrix and **softplus** is applied to its diagonal entries.
- The potential energy V is learned via an MPL that has as inputs the state
- Additional: **friction** matrix learned and rendered positive definite via softplus

Structured Lagrangian Neural Network (SLNN) - Architecture (2)



Structured Lagrangian Neural Network (SLNN) - Results



Structured Lagrangian Neural Network (SLNN) - Comparison LNN vs SLNN vs Baseline

Metric	Baseline	LNN	SLNN
Energy conservation	✗	✓	✓
Friction	✓	✗	✓
correct physics sensitivity	✗	✓	✓
stability	✓	✗	✓
Not-only mechanical systems	✓	✓	✗
Possibility of real inputs	✗	✓	✓
MSE Evolution	21.6	22.9	23.8
MSE Energy	0.7	0.17	0.13

It's possible to add different **modules** to SLNN such as friction to learn different models.

Neuro Probabilistic Model: Motivation

Observation

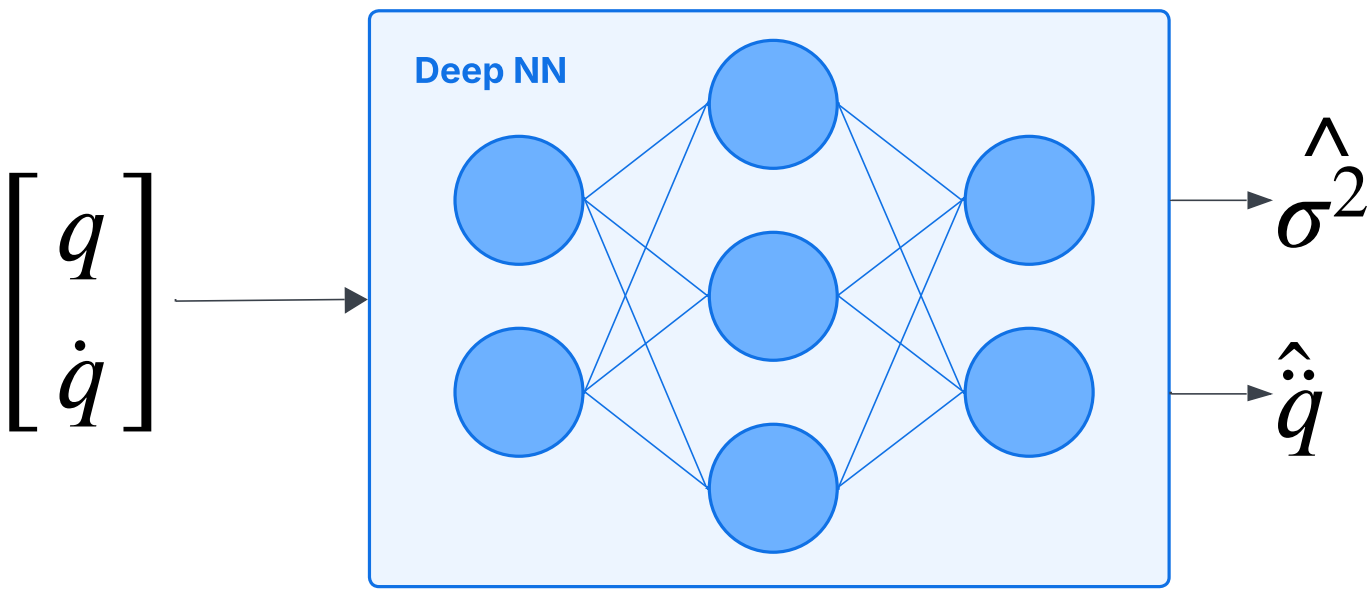
Standard architectures provide **deterministic** predictions, lacking a **stochastic** explanation of confidence.

- **Goal**: Model not only acceleration but also it's variance.
- **Confidence Prediction**:
  - Prediction is associated with a **confidence contour**.
  - A 95% **confidence region** around the expected value computed using the interval: $\ddot{q}_i \in [-2\sigma_i, 2\sigma_i]$
- **Applications**:
  - **Diagnosis**: Enables anomaly detection (alarm triggers) when observations fall outside the predicted variance.
  - **Trajectory generation**: Generate trajectory with an input and evaluate the confidence
  - **Model Predictive Control**: Used to predict the next sliding window with safety margins

Neuro Probabilistic Model: Architecture

To capture aleatoric uncertainty, we modify the Feed-Forward Network (FNN) to output parameters of a probability distribution rather than a single value.

- **Mean** (μ): The expected acceleration.
- **Log-Variance** ($\log \sigma^2$): The estimated uncertainty. Note that we use log for stability.



Neuro Probabilistic Model: Probabilistic Formulation

The model assumes the ground truth acceleration $\ddot{q}$ follows a Gaussian distribution conditioned on the state:

$$p(\ddot{q}|\mathbf{q}) = \mathcal{N}(\ddot{q}|\mu_\theta(\mathbf{q}), \sigma^2_\theta(\mathbf{q}))$$

Training Objective: Negative Log-Likelihood (NLL)

Instead of MSE, we minimize the NLL to learn both the prediction and the uncertainty simultaneously:

$$\mathcal{L}_{NLL} = \frac{1}{2}\log \sigma^2_\theta(\mathbf{q}) + \frac{(\ddot{q}_{gt} - \mu_\theta(\mathbf{q}))^2}{2\sigma^2_\theta(\mathbf{q})} + C$$

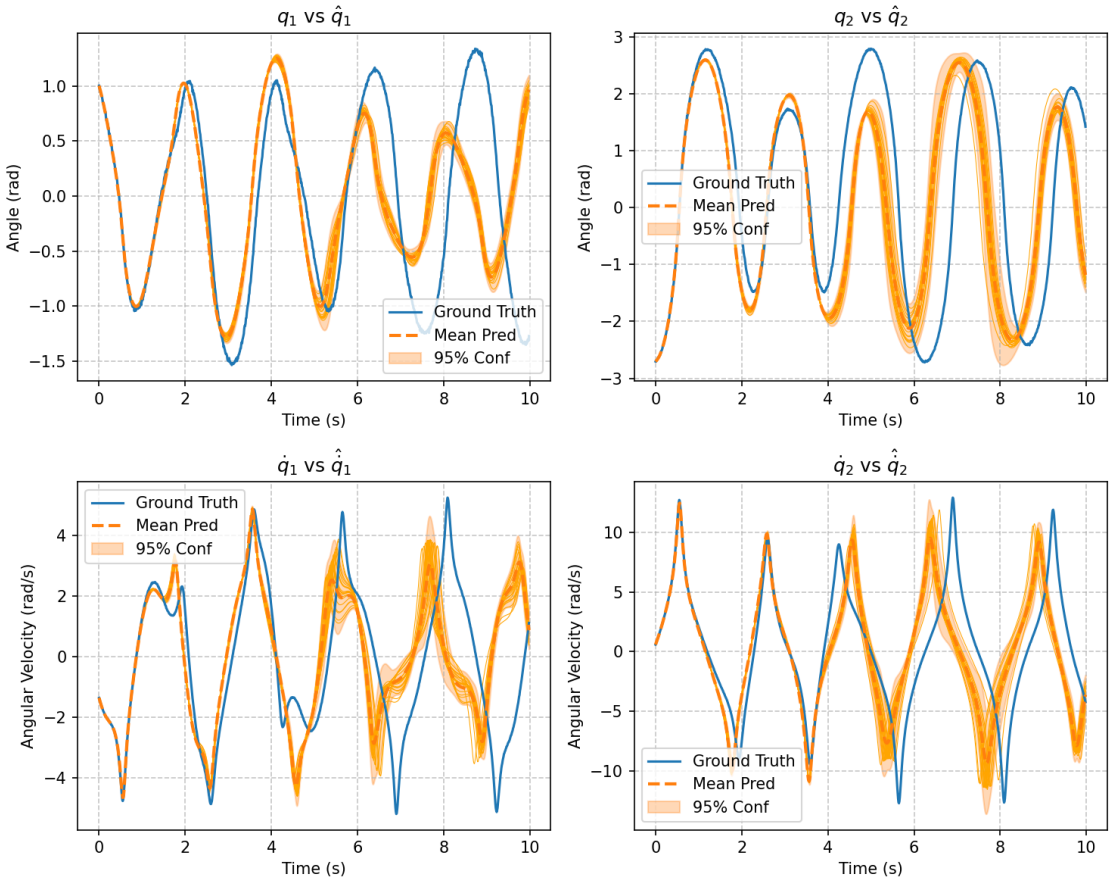
Intuition

The model can reduce the penalty of a high error $(\ddot{x} - \mu)^2$ by increasing its uncertainty σ^2, effectively "admitting" it doesn't know the answer.

Neuro Probabilistic Model: Evolution

Monte Carlo simulation

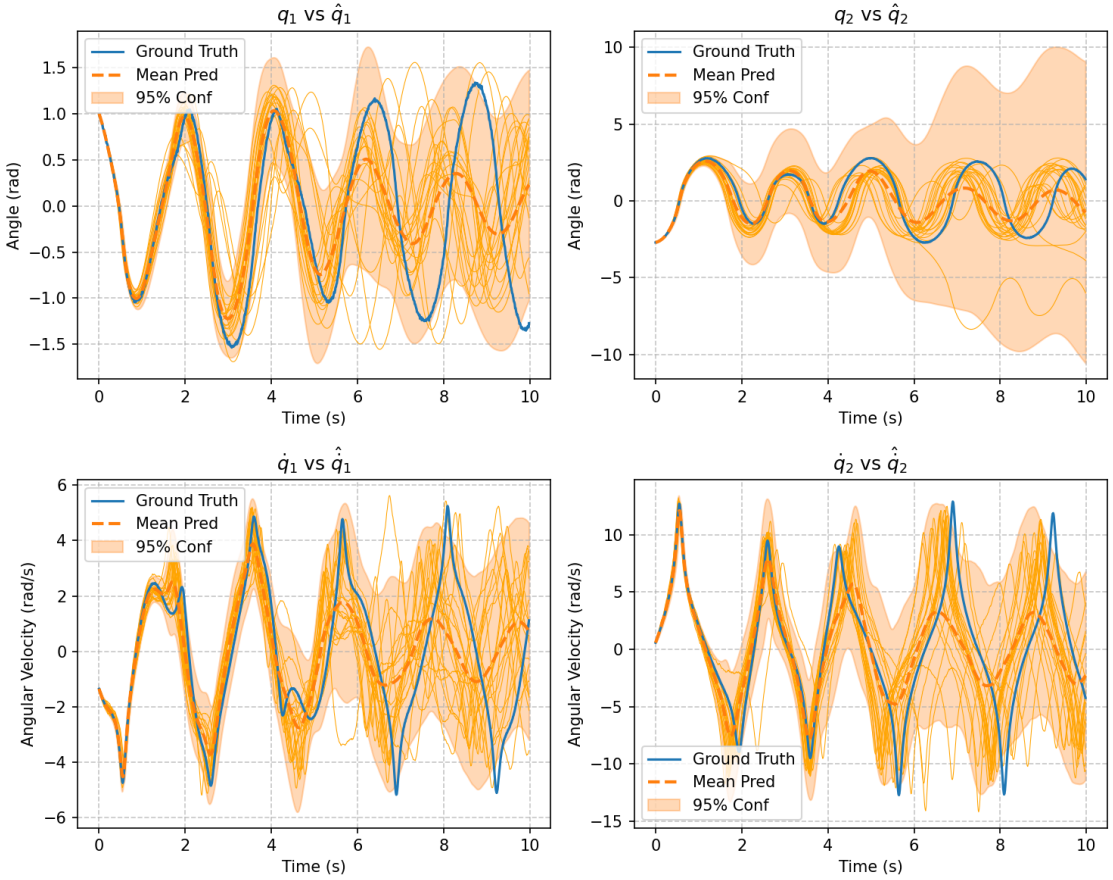
It's possible to use a **Monte Carlo** simulation to predict the confidence region during evolution. Given a **set of pendulums** at every time step the acceleration is sampled according to the model output distribution μ and σ.



Neuro Probabilistic Model: Evolution with dropout

Use of dropout

It is possible to use **dropout** to differentiate the model, so to have an ensamble of models. Note that dropout must remain active also during testing.



Conclusions and Future Works

- **Chaotic Challenge**: It is difficult to predict a chaotic model after several steps.
- **Metrics**: It is better to focus on metrics other than state prediction, as **parameter Identification** and **power spectral density** for instance.

Other points

- **Anomaly Detection** of degraded models or physically wrong ones.
- Developement of a **merged pipeline** that puts together identification and energy conservation.

References


Sam Greydanus, Misko Dzamba, Jason Yosinsky (2019)
Hamiltonian Neural Networks
arXiv (ID: 1906.01563).


Miles Cranmer, Sam Greydanus, Stephan Hoyer, Peter Battaglia, David Spergel, Shirley Ho (2020)
Lagrangian Neural Networks
arXiv (ID: 2003.04630).


Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, David Duvenaud (2018)
Neural Ordinary Differential Equations
arXiv (ID: 1806.07366).


Micheal Lutter, Christian Ritter, Jan Peters (2019)
Deep Lagrangian Networks
arXiv (ID:1907.04490).

Appendix A - Double Pendulum Dynamics

The dynamic model can be expressed in matrix form as:

$$\boxed{M(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + D\dot{\mathbf{q}} + G(\mathbf{q}) = \tau} \tag{6}$$

Inertia Matrix $M(\mathbf{q}) = M(\mathbf{q})^\top \in \mathbb{R}^4 \succ 0$:

$$\begin{bmatrix} (m_1 + m_2)l_1^2 + m_2 l_2^2 + 2m_2 l_1 l_2 c_2 & m_2 l_2^2 + m2 l_1 l_2 c_2 \\ m_2 l_2^2 + m_2 l_1 l_2 c_2 & m_2 l_2^2 \end{bmatrix}$$

Gravity Vector $G(\mathbf{q})$:

$$\begin{bmatrix} (m_1 + m_2) g l_1 s_1 + m_2 g l_2 s_{12} \\ m_2 g l_2 s_{12} \end{bmatrix}$$

Coriolis $\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}})$:

$$\begin{bmatrix} -m_2 l_1 l_2 s_2 (2\dot{q}_1 + \dot{q}_2)\dot{q}_2 \\ m_2 l_1 l_2 s_2 \dot{q}_1^2 \end{bmatrix}$$

Damping Matrix $D = D^\top \succ 0$:

$$\begin{bmatrix} b_1 & 0 \\ 0 & b_2 \end{bmatrix}$$

Note: $s_1 = \sin(q_1), c_1 = \cos(q_1), s_{12} = \sin(q_1 + q_2), c_{12} = \cos(q_1 + q_2)$.